National University of Computer and Emerging Sciences, Lahore

Growise: AI-Powered Dynamic Learning Platform

Muhammad Umair Imran 22L-8370 BS(DS)

Muhammad Shahzad Waris 22L-7530 BS(DS)

Ameer Tufail 22L-7530 BS(DS)

Supervisor: Maryam Nasim

Final Year Project

March 19, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: Growise: AI-Powered Dynamic Learning Platform

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: 14/10/2025

Name: Muhammad Umair Imran

umair

Name: Muhammad Shahzad Waris

ameer

Name: Ameer Tufail

shahzad

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

This project gives a smart learning platform for programmers. It helps them get real problem-solving skills. It does not let them only remember code without thinking. The system first looks at what the learner knows now and his technical past. Then it makes a personal learning road that fits only him. During all the learning time, an AI helper gives help right away, explains things, and brings good materials to make understanding better. After every small part, learners get tested with real coding work that is same as what big company engineers do every day. By mixing learning that changes for each person, non-stop feedback, and real hand practice, the platform helps learners grow true programming power and makes them much more ready for real jobs in software making.

Executive Summary

Today tech world change very very fast. Developer must learn new tool and new way super quick. But old school programming course not good. They only make student remember code word and follow same long plan for everybody. This waste time a lot because some people already know some thing and some want different job. This old way make people not ready for job and they feel bored and stop learn.

So we make one smart learning platform. It teach you how to think like real programmer and how to solve problem, not just remember code like robot. First day the platform give you small test. It check what you know already and what you good at. Then it ask you want learn web or AI or what. After that it make one special plan only for you. You only study what you really need. No waste time.

All the time you study, one AI teacher stay with you. It explain hard thing with easy word. It answer your question fast. It give you video or article that help you right now. After each lesson, you complete projects that feel like real work situations. These projects test whether you understand the concepts and can make good decisions about how to solve problems. You can track your progress on a dashboard that shows you what you're good at, what you need to work on, and how to improve.

By moving away from long, one-size-fits-all courses to a flexible, problem-solving focused approach, this platform bridges the gap between school and real jobs. It helps you grow faster as a developer, teaches you how to think through problems step by step, and gives you skills you can actually use at work. The platform keeps up with how modern software development actually works.

Table of Contents

List of Figures **v**

List of Tables **v**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 1

1.4 Conclusion 2

2 Project Vision **3**

2.1 Problem Domain Overview 3

2.2 Problem Statement 3

2.3 Problem Elaboration 3

2.4 Goals and Objectives 4

2.5 Project Scope 4

2.6 Sustainable Development Goal (SDG) 6

2.7 Constraints 6

2.8 Business Opportunity 7

2.9 Stakeholders Description/ User Characteristics 7

2.9.1 Stakeholders Summary 7

2.9.2 High-Level Goals and Problems of Stakeholders 7

2.10 Conclusion 7

3 Literature Review / Related Work **9**

3.1 Definitions, Acronyms, and Abbreviations 9

3.1.1 Key Definitions 9

3.2 Detailed Literature Review 9

3.2.1 Crio.Do 10

3.2.2 The Odin Project . . . 10
3.2.3 ProjectLearn.io . . . 11
3.2.4 Educative.io . . . 11
3.2.5 Codio . . . 12
3.2.6 Project-Based Learning GitHub Repository . . . 12
3.2.7 Udemy . . . 13
3.2.8 LinkedIn Learning . . . 13
3.2.9 Microsoft Learn . . . 14
3.2.10 Pluralsight . . . 14
3.2.11 CYPHER Learning (with CYPHER Agent) . . . 15
3.2.12 DataCamp . . . 15
3.2.13 freeCodeCamp . . . 16
3.2.14 Udacity . . . 16
3.2.15 CodeCombat . . . 17
3.3 Literature Review Summary Table . . . 17
3.4 Conclusion . . . 19

4 Software Requirement Specifications 20
4.1 List of Features . . . 20
4.2 Functional Requirements . . . 20
4.2.1 Functional Requirements of Programmers / Developers . . . 20
4.2.2 Functional Requirements of Admin . . . 21
4.2.3 Functional Requirements of System . . . 21
4.3 Quality Attributes . . . 21
4.4 Non-Functional Requirements . . . 22
4.4.1 Performance . . . 22
4.4.2 Reliability . . . 22
4.4.3 Usability . . . 22
4.4.4 Reusability . . . 22
4.4.5 Extensibility . . . 22
4.5 Assumptions . . . 22
4.6 Use Cases . . . 23
4.6.1 User Login . . . 23
4.6.2 Track Selection . . . 24
4.6.3 Skill Assessment . . . 25

4.6.4 AI Path Generation . . . 26
4.6.5 Agentic RAG Assistance . . . 27
4.6.6 Project-Based Assignment . . . 28
4.6.7 Admin Login . . . 28
4.6.8 Manage Learning Paths . . . 29
4.6.9 Manage RAG Knowledge Base . . . 29
4.6.10 Monitor User Progress . . . 30
4.6.11 Learning Path Module Access . . . 31
4.7 Hardware and Software Requirements . . . 31
4.7.1 Hardware Requirements . . . 31
4.7.2 Software Requirements . . . 31
4.8 Graphical User Interface . . . 32
4.9 Database Design (if required; this means if you are using noSQL, you will not provide ER but the other design element and data dictonary should still be there. Explain and elaborate your DB design) . . . 35
4.9.1 ER Diagram . . . 35
4.9.2 Data Dictionary . . . 37
4.10 Risk Analysis . . . 45
4.10.1 Technical Risks . . . 45
4.10.2 Business Risks . . . 45
4.10.3 User Risks . . . 46
4.10.4 Security Risks . . . 46

5 High-Level and Low-Level Design 47
5.1 System Overview . . . 47
5.1.1 Learning Management and Modules . . . 47
5.1.2 Personalized Learning Paths . . . 47
5.1.3 Agentic RAG Mentor (Context-Aware Assistance) . . . 47
5.1.4 Assessment and AI Software Engineer Evaluation . . . 47
5.1.5 Progress Insights and Feedback . . . 47
5.1.6 High-Level Technology View . . . 48
5.2 Design Considerations . . . 48
5.2.1 Assumptions and Dependencies . . . 48
5.2.2 General Constraints . . . 49
5.2.3 Goals and Guidelines . . . 49

5.2.4 Development Methods . . . 49
5.3 System Architecture . . . 50
5.3.1 High-Level System Decomposition . . . 51
5.3.2 Major System Responsibilities . . . 51
5.3.3 Component Roles and Responsibilities . . . 51
5.3.4 Component Collaboration and Data Flow . . . 52
5.3.5 Architectural Rationale . . . 53
5.3.6 External System Integration . . . 53
5.3.7 Subsystem Architecture . . . 53
5.4 Architectural Strategies . . . 54
5.4.1 Using External AI APIs . . . 54
5.4.2 Using RAG for Smart Help . . . 54
5.4.3 Choosing Modern Technology . . . 55
5.4.4 Using Microservices . . . 55
5.4.5 API-First Design . . . 55
5.5 Domain Model/Class Diagram . . . 55
5.6 Policies and Tactics . . . 57
5.6.1 Codebase Organization . . . 57
5.6.2 Development Tools . . . 58
5.6.3 Coding Guidelines . . . 58
5.6.4 Testing Strategies . . . 58
5.6.5 Documentation Practices . . . 58
5.7 Conclusion . . . 58

6 Implementation and Testing **60**
6.1 Implementation . . . 60
6.1.1 Learning Track and Module System . . . 60
6.1.2 AI-Powered Skill Assessment Module . . . 60
6.1.3 AI Learning Path Generation Module . . . 61
6.1.4 RAG Implementation for Integrated Learning Sources Context . . . 61
6.2 Conclusion . . . 62

7 Conclusions and Future Work **63**
7.1 Project Summary and Achievements . . . 63
7.2 Key Findings and Results . . . 63

7.3 Project Scope Coverage and Objectives Achievement 63
7.4 Challenges Faced and Limitations 63
7.5 Future Work and Recommendations 64
7.5.1 Phase 2 Development Plan (FYP-2) 64
7.5.2 Long-term Recommendations 64
7.6 Final Remarks 65

List of Figures

2.1 SDG 4 Quality Education – this is the goal GrowWise help. 6

4.1 Login screen for users to enter credentials and access the system. 32
4.2 Track selection screen where users choose their learning path. 33
4.3 Skill assessment page for evaluating users' technical and problem-solving abilities. . . 33
4.4 Assessment report showing the user's skill level and performance summary. 33
4.5 Personalized roadmap showing the AI-generated learning journey and module sequence. 34
4.6 Learning module interface with study content and an integrated AI chatbot. 34
4.7 AI software engineer screen that assigns projects and evaluates user responses. . . . 34
4.8 Insights dashboard with user progress analytics. 35
4.9 Advanced insights with AI evaluation and performance graphs. 35
4.10 GrowWise Entity-Relationship Diagram with all entities, keys, and relationships. . . 36

5.1 GrowWise architecture overview—showing interaction between frontend, backend, AI services, and database. 50
5.2 GrowWise system architecture: frontend, backend, AI, and database. 56
5.3 GrowWise – View 2: AI Services & Knowledge Management, highlighting components like the RAG knowledge base, contextual assistant, path generator, and AI mentor for adaptive, context-aware learning. 56
5.4 GrowWise – View 3: Project Workflow & Evaluation, showing how projects are assigned, submitted, and AI-evaluated with scoring and feedback. 57

List of Tables

3.1 Summary of Learning Platforms Reviewed 18

4.1 Use Case: User Login 23
4.2 Use Case: Track Selection 24
4.3 Use Case: Skill Assessment 25
4.4 Use Case: AI Path Generation 26
4.5 Use Case: RAG AI Assistance 27
4.6 Use Case: Project Assignment 28
4.7 Use Case: Admin Login 28
4.8 Use Case: Learning Path Management 29
4.9 Use Case: RAG Knowledge Management 29
4.10 Use Case: User Progress Monitoring 30
4.11 Use Case: Module Access 31
4.12 Test case for User 38
4.13 Test case for LearningTrack 38
4.14 Test case for LearningModule 39
4.15 Test case for UserTrackSelection 39
4.16 Test case for UserProgress 40
4.17 Test case for SkillAssessment 40
4.18 Test case for PersonalizedLearningPath 41
4.19 Test case for UserInsights 41
4.20 Test case for RAGKnowledgeBase 42
4.21 Test case for AIMentorInteraction 43
4.22 Test case for ProjectAssignment 44
4.23 Test case for UserProjectAssignment 44
4.24 Test case for ProjectSubmission 45

Chapter 1 Introduction

Since ChatGPT launched in 2022, the way we build software has changed dramatically. Programmers now need to constantly learn new technologies and stay flexible. But traditional programming courses, with their long and rigid lesson plans, can't keep up with how fast things change in tech today.

This project make one smart learning app. First it check what the programmer already know. Then it make special study plan only for him. Inside got one AI teacher that give good feedback every time. Also got test that feel like real company job. It help you really understand thing and really can use it. The other page of this report tell how we design it, how we build it, how we test it, and what result we get.

1.1 Purpose of this Document

This paper explain how we design, build and test one smart learning platform for programmer. We want change how people learn code by make it faster, more personal and teach real problem solving not just remember code. Old course give same long lesson to everyone but our platform first test what you already know then make special plan only for you and your job dream. Main question is can AI make learning personal and help programmer learn super fast by focus on solve problem. This project show real working app that do it. It test your skill with AI, give personal help from AI teacher and check your code like senior developer. This paper talk everything: how we study problem, design system, build it, test it, what work good, what not good and what we do next.

1.2 Intended Audience

This report is for two type people. First is professor, supervisor and examiner who check our project good or not and if it new idea. Second is beginner programmer or self-study coder who want learn fast and better. We write with some hard detail for teacher but also explain easy so normal programmer can understand and use our idea. This way the report work for school mark and also help real coder in future.

1.3 Definitions, Acronyms, and Abbreviations

List Of Abbreviations: SDG: Sustainable Development Goal

FYP: Final Year Project

MVP: Minimum Viable Product

UI: User Interface

UX: User Experience

Agile: Agile Development

SCRUM: Scrum Development

REST: Representational State Transfer

ORM: Object-Relational Mapping

CRUD: Create, Read, Update, Delete

API: Application Programming Interface

1.4 Conclusion

GrowWise is one smart AI learning app for programmer. It give personal study plan, AI teacher help every time, and real hard coding test. It fix old school problem because old way only make people remember code but not think. This report tell the main idea, what we need, and how we design the system. Now we got good start but still not finish. We will keep build and make it better in FYP-2 until GrowWise become full complete app.

Chapter 2 Project Vision

This chapter explain the big idea behind our AI learning platform for programmers. We talk about what is wrong with normal coding courses right now, what we want to fix, our main goals, and what exactly our project will do and will not do. After you read this part, you will understand why we make this system and how it can change the way people learn programming today.

2.1 Problem Domain Overview

GrowWise want fix big problem in how people teach programming now. Most app give same lesson to everybody. The course very hard to change and not look at your level or how you think. This old way make learning slow, waste time on thing you already know, and not teach you how to solve problem good.

Now tech world change super fast. Programmer need system that teach how to think and solve problem, not just remember code like robot. Most platform only care about you remember syntax good. They not teach you how to choose right way or understand big picture.

GrowWise fix all this problem by use AI. First it test what you already know. Then it make one special learning road only for you. Because of this you only study important thing, learn how to solve problem good, become better super fast, and ready for real programmer job.

2.2 Problem Statement

Normal learning platform very slow and same for everybody. They not look at your real skill level or how you think. Now we need more teaching that help developer learn how to solve problem, not just remember tool or code stack.

2.3 Problem Elaboration

Now day learning app have many problem together. These problem stop people become good programmer and ready for job. Below we list the main problem this project want fix:

1. No Personal Plan: Most platform give same lesson to all people. But everyone got different skill and learn different way. So same plan for all make people bored and not good.
2. Wrong Level: Many time you study thing you already know or thing too hard. No test first, so you waste time and learn slow.

3. Course Too Long and Hard to Change: Old course take many week or month. Tech change very fast but course still old. You finish late already.
4. Only Remember Code: Teacher only make you remember syntax and tool. They not teach how to think and solve problem. When new tool come you stuck.
5. Not Enough Real Work: Best learn is do real project. But many platform give only small easy task. No hard real job challenge. So you not ready for company work.

2.4 Goals and Objectives

Main goal of this project is make one smart learning app. It teach you how to solve problem real good, not just remember code and tool. We want give programmer fast, only-for-you learning with AI help all the time.

Our big objectives are:

1. Personal Study Plan: System first test what you know. Then make special plan for you. It look what you strong, what you weak, and what job you want.
2. Start Right Level: You begin not too easy not too hard. No waste time on thing you already know. Also not cry because too difficult.
3. Fast and Can Change: No long boring course. Short lesson that change when you learn fast or slow.
4. Teach How to Think: We teach you think like real engineer. Not only remember syntax. We teach logic and system design.
5. Real Job Test: Give you hard project same like company work. You practice real thing, not baby task.
6. AI Teacher Very Smart: Got one AI mentor like good friend. It find best video or article for you, answer question, and help you every minute.

2.5 Project Scope

This project is about make one smart learning app for coder. It change the lesson for every person. It look what you already know, how fast you learn, and what job you want like web or AI. Then it give you only the right thing to study. The platform goes beyond traditional courses by using AI mentors, thorough skill tests, and a focus on teaching problem-solving methods.

The platform enables users to:

- Execute secure registration and authentication protocols as authenticated learners.
- Take an initial test that checks both your technical knowledge and problem-solving skills.
- Designate preferred learning specializations (e.g., Web Development, AI Applications, Cybersecurity, etc.).
- Get a custom learning plan created by AI based on your test results and what tech field you chose.
- Use an AI Mentor that gives you explanations when you need them, instant feedback on your work, and suggests helpful learning materials as you go through each lesson.
- Work on realistic project tests that feel like actual programming jobs and check how well you solve problems and design systems.
- Let you see your progress on one personal dashboard. It show what you good at, what still weak, and what you should do next to become better.

We build this platform use FastAPI for back end, React.js for front end look nice, and Python for AI part. The AI teacher use RAG to find good video or article and give smart answer. We work Agile way, mean we build little little, test every time, fix fast, and ask user what they like or not like all the time.

Project give you this thing:

- One working web app that you can open in browser and use the smart AI learning.
- Full paper that explain how system work, why we choose this way, how we code it, and how we test.
- Easy user guide book that teach you how to use the app step by step.

But this project NOT do this:

- Connect with other big school system (LMS).
- Make phone app for Android or iPhone.
- Make super big server for many thousand user. Only small test server is enough.

Because we make clear what we do and what we not do, now we know exactly what to build and can finish on time.

2.6 Sustainable Development Goal (SDG)

Our project GrowWise is AI learning app for programmer. It match with United Nations SDG number 4 that is Quality Education. This goal say everyone must get good study and can learn all life. GrowWise help this because it use AI to give personal lesson for every people. Some people slow some people fast no problem. The app change for you. It give fast feedback and real project test so you learn skill that company want now. Also anyone can use it free and study when they want. So it make study fair for poor or busy people and help them get good tech job. This way GrowWise help SDG 4. 2.1.

Figure 2.1: SDG 4 Quality Education – this is the goal GrowWise help.

2.7 Constraints

When we make this AI learning platform we got some problem and limit:

- Computer not strong enough: AI like RAG and smart teacher need big power and memory. Our laptop or small server sometime slow or crash.
- Time very short: This is only Final Year Project so we got few month only. We cannot make super smart system or add too many new thing.
- Many part hard to connect: We got test AI, mentor AI, and chat AI. Make all work together smooth is difficult and got bug many time.
- Must have internet all time: Everything use online AI and search. No wifi mean app stop work.
- Only for few people now: This version just for test with small user. We not make big server for

thousand people same time. That is for later.

2.8 Business Opportunity

GrowWise not only smart learning app but also big money chance. Many people, company and school can pay for it. Normal course not personal and not teach real thinking. GrowWise do both so it got special place in market. Many programmer want fast good skill and company want good worker quick. GrowWise can sell to them and make good business.

2.9 Stakeholders Description/ User Characteristics

This part talk who will use GrowWise and what they want. We know them good so we can make app they like.

2.9.1 Stakeholders Summary

GrowWise got two big group of people:

- Normal Programmer / Junior Developer: This is main user. They new or little bit know code. Got student, people change job to tech, self study coder, or junior want become better. They want app that know their level, teach slow or fast same them, and teach how to think not remember code. They want ready for real company job.
- Tech Company / Boss: Big company or small team that want train worker fast. They want see who learn good who not, give same training to many people, and make sure worker can do real project. They like if app give report and certificate.

2.9.2 High-Level Goals and Problems of Stakeholders

- Junior Programmer: Want personal plan, learn problem solving, not boring remember thing.
- Company: Want worker learn quick, can check progress, and worker ready for real work fast.

Because we know what they need, we build GrowWise that make both group happy.

2.10 Conclusion

Chapter 2 tell about GrowWise app. We talk what problem normal coding course have and why it bad. We say what we want fix and what our goal is. We tell what we will do and what we cannot do because

time short. Also we say this app can make money later. Now everything clear so next chapter we talk how we design and build the real app.

Chapter 3 Literature Review / Related Work

This chapter look at other app and tool already have. We study many platform that do personal learning, give real project, or use AI to teach programmer. All this old work give us good idea for GrowWise. We take best part from them and make better by add RAG for smart answer, AI that check code, and special plan only for junior developer who want learn real skill.

3.1 Definitions, Acronyms, and Abbreviations

This part explain short word and hard word we use in this chapter. We write what they mean so you not confuse when read.

3.1.1 Key Definitions

Adaptive Learning: Smart teaching way that change lesson, speed and style same time when it see how you learn.

Agentic AI: AI that can think and do work alone like human, no need people tell every thing.

Approach-Oriented Learning: Teach you how to think and solve problem, not just remember code line.

Learning Path: Step by step plan what you study, what exercise you do, what test you take.

Micro-Learning: Very small short lesson, only few minute, easy to finish fast.

Personalized Learning: Lesson made only for you, match your speed and what you like.

RAG (Retrieval-Augmented Generation): Smart AI that search internet or book first, then give correct answer.

Skill Assessment: Test to check what you already know good and what you still weak.

3.2 Detailed Literature Review

This Chapter provides an overview of related applications which inpisered the idea of GroWise. A carefully conducted critical analysis have been done for the following application and their relationship with proposed work.

3.2.1 Crio.Do

3.2.1.1 Summary

Crio.Do [?] is online learning site. It help student jump from school code to real company job. They work with big company like CRED and give you real project that company actually use. Not just talk theory. You really build app. Good for student and junior developer. After finish you got nice portfolio and ready for job.

3.2.1.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Give real job experience. Work with real company so people trust it. Teach skill that company want so easy get job. Weaknesses: Only teach few technology. Beginner can feel too hard. Project take many time, busy people cannot finish.

3.2.1.3 Relationship to Proposed Work

GrowWise learn from Crio.Do because Crio.Do good at real project. But GrowWise better because it make personal plan for every student with AI. It change lesson for you and teach many more technology.

3.2.2 The Odin Project

3.2.2.1 Summary

The Odin Project [?] is free open website teach full-stack web coding. It give full plan from beginner to finish. You learn HTML, CSS, JavaScript, Git, Node.js, database and more. It not give easy step-by-step video. It make you build real app alone so you learn how to think and fix problem. Everything free and people in community help each other. Good for self study person who want become job ready web developer.

3.2.2.2 Critical Analysis (Strengths and Weaknesses)

Strengths: 100Weaknesses: You study alone so lazy people stop easy. Mostly only text, not many video or interactive. Not feel like real company work.

3.2.2.3 Relationship to Proposed Work

GrowWise like The Odin Project because both teach by build real thing. But GrowWise better because got AI teacher give personal help, change plan for you, and give feedback fast every time.

3.2.3 ProjectLearn.io

3.2.3.1 Summary

ProjectLearn.io [**?**] is one learning site. It teach programming by make real app. It got many tutorial and resource for different tech like web, mobile, game and machine learning. Not small exercise only. You build full app from beginning to end. You see how all part work together. Tutorial sort by tech and hard level. Easy for you find right project for your skill and like. You build good portfolio while learn. Good for student who want show real work not just talk theory.

3.2.3.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Many project in different area. Help you make portfolio. Got easy and hard level for all people. Weaknesses: Some tutorial good some bad because many people make. No personal plan for you. Not much interactive thing. Only focus on project.

3.2.3.3 Relationship to Proposed Work

GrowWise like ProjectLearn.io because both teach by real project. But GrowWise better. It got better order and personal lesson for every student. Use AI to change for you and give fast feedback.

3.2.4 Educative.io

3.2.4.1 Summary

Educative.io [**?**] is learning site for programmer who already know some code. It teach hard thing like system design, interview question, backend and big company topic. Best part is you code inside browser, no need open other program. AI look what you do and give personal plan and fast feedback. Good for medium or good developer who want pass big company interview.

3.2.4.2 Critical Analysis (Strengths and Weaknesses)

Strengths: You can code right in website. Got many course for interview and system design. AI make plan only for you. Weaknesses: Must pay money for good course. Not much full-stack basic thing. Too hard for total beginner.

3.2.4.3 Relationship to Proposed Work

GrowWise take good idea from Educative.io like code in browser and AI personal plan. But GrowWise focus more on beginner and junior developer. We give full path from start and use more AI to help new

people.

3.2.5 Codio

3.2.5.1 Summary

Codio [?] is cloud learning app. It give real coding place inside browser. Got many course like AI, data, security and full-stack. Best thing is you use same tool like real company: GitHub, CI/CD pipeline. No need install anything on your laptop. Good for school and company training because student learn real work style fast.

3.2.5.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Real tool same as company. No setup problem. Many different course. Weaknesses: Too many thing, beginner can confuse. Must pay for good course. Not big community, hard find friend help.

3.2.5.3 Relationship to Proposed Work

GrowWise like Codio because both give real coding place. But GrowWise easier for beginner. We use AI to change lesson for every student and give help fast. GrowWise not too hard and make personal plan so new developer learn better.

3.2.6 Project-Based Learning GitHub Repository

3.2.6.1 Summary

This GitHub repo [?] is big free collection that many people add. Got hundred project tutorial for all kind tech: web, mobile, data science, many language. Everything sort by language so easy find. You build full real app from zero and got nice portfolio. Good for people who like study alone but no plan and no teacher help.

3.2.6.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Free, many project, all language, help make good portfolio. Weaknesses: No clear order, beginner easy lost. Some tutorial good some bad. Only text, no code place inside, no fast help when stuck.

3.2.6.3 Relationship to Proposed Work

GrowWise like this repo because both teach by build real project. But GrowWise better. We give clear step plan, AI teacher help every time, personal lesson, and project become harder same your skill. You

never lost and learn faster.

3.2.7 Udemy

3.2.7.1 Summary

Udemy [**?**] is very big online course shop. Got more than 200,000 course about code, web, data science and many thing. Any teacher can sell their video course. Mostly video lesson, some file to download and little bit coding task. You buy one course cheap, study any time you want. Good for beginner and pro because price low and many choice. But quality not same, some course very good some not good.

3.2.7.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Super many course, cheap price, can study when you free, easy find course by rating. Weaknesses: Course not change for you, no personal plan. Almost no fast feedback. Many course only watch video, no real project. People start but not finish because no one push.

3.2.7.3 Relationship to Proposed Work

Udemy good because got many topic and cheap. But GrowWise much better. We use AI make plan only for you, give fast feedback, real hard project, and AI teacher help every time. GrowWise teach you how to think and solve problem, not just watch video and forget.

3.2.8 LinkedIn Learning

3.2.8.1 Summary

LinkedIn Learning [**?**] is big video course site for job people. It teach tech, business and creative skill. Best part is it connect with your LinkedIn profile. When you finish course, certificate show on your profile so boss can see you learn new thing. It look at your job and skill then suggest course that help your career. Teacher are real expert from company. Good for worker who want look good to boss and get better job.

3.2.8.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Good quality video, certificate show on LinkedIn, suggest course match your job, many career path. Weaknesses: Mostly only watch video, not much real coding or project. Personal plan not very smart, no AI change for you. No hard test like real work.

3.2.8.3 Relationship to Proposed Work

LinkedIn Learning good because got nice course and help career. GrowWise learn from it about career plan. But GrowWise much better for real programmer skill. We give AI teacher all time, real hard project, fast feedback, and change lesson every minute for you. GrowWise make you really can code good, not just watch video and get paper.

3.2.9 Microsoft Learn

3.2.9.1 Summary

Microsoft Learn [**?**] is free learning site from Microsoft. It teach Azure, Microsoft 365, Power Platform and other Microsoft thing. You can play with real cloud tool in sandbox, no need buy anything. Got clear path for job like developer or admin. It look what you know and suggest next lesson. Everything free, got lab, interactive tutorial and small test. Good if you want work with Microsoft tech.

3.2.9.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Free, real Microsoft tool to play, good order path, change little bit for your level. Weaknesses: Only teach Microsoft thing, not other tech. Mostly small exercise, not big real project. Not check how you think or design system, only check you finish task or not.

3.2.9.3 Relationship to Proposed Work

GrowWise learn from Microsoft Learn about personal suggest and free lab. But GrowWise work with any tech, not only Microsoft. We focus on real problem solving, how you think, system design and big project. AI mentor help you every step and give smart advice fast. GrowWise make you real strong programmer, not just Microsoft worker.

3.2.10 Pluralsight

3.2.10.1 Summary

Pluralsight is pay every month learning site for IT and coder people. Got thousand video course about code, cloud, security and more. Best thing is Skill IQ test. It check how good you are in one tech fast. Then it make personal plan for you, show what you weak and what course to watch. Company love it because can see worker skill number and train team easy.

3.2.10.2 Critical Analysis

Strengths: Very good test system, make real personal plan. Many course, good for company training. Show you compare with other programmer. Weaknesses: Mostly only video, not much real coding inside site. Not teach how to think design or solve big problem. No fast feedback when you code project.

3.2.10.3 Relationship to Proposed Work

GrowWise and Pluralsight both start with skill test and give personal path. But GrowWise better because not just video. We give real project, teach how to think and design system, and got AI mentor help you every minute with smart answer. GrowWise make you real problem solver, not just watcher.

3.2.11 CYPHER Learning (with CYPHER Agent)

3.2.11.1 Summary

CYPHER Learning is big company learning system (LMS) for train many worker same time. It got one AI call CYPHER Agent. This AI look your job, your skill and what company want, then make personal lesson for you. It find what you weak and give right course fast. Good for big company because can track all worker and finish training easy.

3.2.11.2 Critical Analysis

Strengths: AI really work good for personal plan. Can teach thousand people different thing same time. Save boss time. Weaknesses: Make for normal worker training, not special for programmer. Not teach how to think code or design system. Content too general, not deep tech.

3.2.11.3 Relationship to Proposed Work

CYPHER show that AI teacher inside big system can work nice. GrowWise use same idea with RAG Mentor. But GrowWise only for programmer. Our AI give fast help like senior developer, check your project real hard, and teach you how to think and solve problem proper way.

3.2.12 DataCamp

3.2.12.1 Summary

DataCamp is learning site only for data science, analytics and AI. It give short video then you code right away inside browser, no install anything. You use Python, R, SQL real time. It check your code fast

and give feedback same minute. AI look how you do and change next lesson easy or hard for you. Also suggest what course good for your job dream. Good for beginner to pro in data field.

3.2.12.2 Critical Analysis

Strengths: Real coding inside site, feedback super fast, AI change plan for you, good for learn data skill quick. Weaknesses: Only data science and Python/R, nothing about web or security. Test only small exercise, not big real project. Not check how you think or design system.

3.2.12.3 Relationship to Proposed Work

GrowWise like DataCamp because both give fast feedback and change lesson for you. But GrowWise teach all tech, not only data. We give big real project, test how you think and design like senior developer. GrowWise make you full programmer, not just data person.

3.2.13 freeCodeCamp

3.2.13.1 Summary

freeCodeCamp is non-profit website teach coding 100

3.2.13.2 Critical Analysis

Strengths: Totally free, give real project, certificate real, now got AI help, big community. Weaknesses: You study alone, beginner easy lost, no personal plan, no one push you finish, sometime stuck long time.

3.2.13.3 Relationship to Proposed Work

GrowWise and freeCodeCamp both love real project and want free education. But GrowWise fix the problem. We give personal plan with AI, AI mentor help every minute, give fast feedback, and push you finish. GrowWise make you never lost and learn faster than freeCodeCamp.

3.2.14 Udacity

3.2.14.1 Summary

Udacity sell expensive course call Nanodegree. It teach AI, cloud, data science and more. Big company like Google, Amazon help make the course so content very new and real. You do many hard project like real job and build good portfolio. Got real human mentor check your work and give feedback. After finish you ready for high pay tech job. But price very high.

3.2.14.2 Critical Analysis

Strengths: Course very good and match company want. Many real project. Human mentor help you. Got deadline so you finish. Weaknesses: Super expensive. Mentor is human so sometime slow answer. Plan same for everybody, not change when you good or bad fast.

3.2.14.3 Relationship to Proposed Work

Udacity good because teach real project and make you job ready. GrowWise do same but cheaper and better. We not use slow human mentor. We use fast AI mentor 24/7 with RAG and smart check. AI change plan every day for you. You pay less but learn faster and more personal.

3.2.15 CodeCombat

3.2.15.1 Summary

CodeCombat is game that teach you code. You play fantasy game and write real Python or JavaScript to move hero and win fight. Very fun, especially for kid and total beginner. When you stuck, AI give hint fast. Level start easy then become hard. Good way to learn basic coding without boring.

3.2.15.2 Critical Analysis

Strengths: Super fun, you want play more. Feedback come same second. AI hint help when stuck. Good for young people or first time coder. Weaknesses: Only game style, not teach real big project or how to design system. Problem always got one correct answer, real job not like that. Not good for medium or pro developer.

3.2.15.3 Relationship to Proposed Work

GrowWise like CodeCombat because both use AI to give fast help and keep you not stuck. But GrowWise for real programmer, not kid game. We give big real project, AI check how you think and design like senior engineer. GrowWise make you job ready, not just play game.

3.3 Literature Review Summary Table

The following table provides a comprehensive summary of the learning platforms analyzed in this literature review, highlighting their key features, relevance to the proposed GrowWise system, and identified limitations.

Table 3.1: Summary of Learning Platforms Reviewed

Platform	Key Features	Relevance to GrowWise	Identified Limitations
Crio.Do [?]	Project-based learning with real-world company projects	Provides insight into hands-on skill development and project experience	Limited to certain tech stacks; time-intensive; may overwhelm beginners
The Odin Project [?]	Free, open-source full-stack web development curriculum	Emphasizes learning by doing and portfolio building	Self-paced nature may reduce accountability; text-heavy; limited industry exposure
ProjectLearn.io [?]	Curated project tutorials across multiple domains (web, mobile, ML, game dev)	Focus on project-based learning and portfolio creation	Quality varies across creators; lacks personalized paths; limited interactive features
Educative.io [?]	Interactive coding courses with AI personalization	Provides hands-on coding practice and adaptive learning paths	Subscription required for some courses; limited full-stack coverage; better for intermediate learners
Codio [?]	Real-time coding environments with GitHub	Offers immersive, practical coding experience and workflow integration	Can be complex for beginners; subscription required; limited community support
Project-Based Learning GitHub Repository [?]	Curated tutorials for multiple languages and frameworks	Encourages learning by doing and project portfolios	Lacks structured learning paths; quality varies; limited interactivity
Udemy [?]	Massive course catalog across various technologies	Wide accessibility; self-paced learning	Static content; limited personalization; minimal real-time feedback
LinkedIn Learning [?]	Professional courses with LinkedIn integration	Structured career-oriented learning paths	Mostly video-based; minimal coding practice; limited project validation
Microsoft Learn [?]	Personalized learning paths with hands-on labs	Interactive exercises and adaptive recommendations	Focused on Microsoft technologies; limited assessment of problem-solving or architecture skills
Pluralsight [?]	Skill IQ assessment, adaptive testing, personalized learning paths, enterprise training	Assessment-first approach, skill-level matching, industry benchmarking	Video-based delivery, limited interactive practice, focus on skill acquisition over architectural evaluation
CYPHER Learning [?]	Agentic AI integration, adaptive learning, enterprise LMS, AI skills management	Personal learning agent validation, agentic AI philosophy, adaptive learning capabilities	Enterprise focus, generalized content, limited specialized developer training
DataCamp [?]	Interactive coding exercises, micro-assessments, AI-recommended paths, real-time adaptation	Interactive learning model, continuous assessment, adaptive pacing	Domain-specific focus, limited architectural reasoning, emphasis on code mastery over problem-solving approach
freeCodeCamp [?]	Free curriculum, real-world projects, community-driven learning, AI tutoring integration	Project-based learning, accessibility, real-world application	Self-paced nature, limited personalized guidance, overwhelming for beginners, lack of accountability
Udacity [?]	Nanodegree programs, industry partnerships, project-based learning, human mentor support	Industry-ready focus, project-intensive learning, career alignment	High cost, human-gated feedback, limited scalability, fixed learning paths
CodeCombat [?]	Gamified learning, interactive coding, AI-driven hints, immersive environments	Engagement strategies, interactive environments, AI-driven guidance	Gamification limitations, simplified complexity, limited real-world architectural assessment

3.4 Conclusion

In this chapter, we looked at many learning platforms including Crio.Do, The Odin Project, ProjectLearn.io, Educative.io, Codio, GitHub projects, Udemy, LinkedIn Learning, and Microsoft Learn. These platforms focus on learning by doing projects, hands-on practice, and coding exercises, which are great for building skills and portfolios. But most don't offer personalized learning that changes as you go, AI mentors, instant feedback on how well you understand concepts, or custom learning paths based on your current level and what you want to learn.

This review shows what's missing in current platforms something GrowWise can fill. GrowWise combines personalized AI help, smart resource searching, and project testing to teach you how to think through problems. What we learned from these platforms helps us design GrowWise to fix their problems while improving your critical thinking, problem-solving, and real-world programming skills.

Chapter 4 Software Requirement Specifications

This chapter explains the main features of the project. It also describes what the system should do, how it should work, the use cases, and the possible risks involved.

4.1 List of Features

GrowWise app got this main thing:

- Sign up and login safe way. You make profile easy.
- Choose what you want learn like AI, Web, Game or other track.
- First day got skill test. App know what you good and what you weak.
- App make special study plan only for you from your test and what track you pick.
- Every lesson got AI teacher. It explain, give hint, show video or article when you need.
- AI give you real project. After you finish, AI check your code, your thinking way, your design and give feedback like senior developer.
- You see your progress page. It show what you strong, what still bad, and what to do next.
- Admin can add new track, put new file for AI to read, and fix thing when broken.

4.2 Functional Requirements

This part tell what the app can do for each person.

4.2.1 Functional Requirements of Programmers / Developers

Programmer user can do this:

- Make new account and login.
- Pick what track want learn like AI, Web or Game.
- Do first skill test so app know your level.
- Get personal study plan only for you.
- Open each lesson and talk with AI teacher, see example and video.
- Do small task or mini project in every lesson.
- See progress and get feedback what good what bad.

- Finish one big final project. AI check how you think and design.

4.2.2 Functional Requirements of Admin

Admin can do this:

- Make new track or edit old track.
- Add new lesson, video, article.
- Put new file inside RAG so AI become smarter.
- Look all user path and fix if something wrong.

4.2.3 Functional Requirements of System

The app itself can do this automatic:

- Keep all user account safe.
- Make personal plan for every new user.
- Give AI teacher help in every lesson.
- Watch your progress and give feedback fast.
- Give big project and check your code with AI like real boss.
- Keep all file and knowledge inside RAG for AI to use.

4.3 Quality Attributes

GrowWise made to be good and easy. Main good point are:

- Reliability: AI teacher and checker always give right answer. App not crash easy.
- Usability: Everything look simple and clear. You find lesson, video, feedback very easy.
- Maintainability: Code clean and got good order. Later easy add new thing or fix bug.
- Performance: App fast, even many people use same time still smooth.
- Flexibility: Easy add new track, new file for AI, or new feature later.

4.4 Non-Functional Requirements

This section describes the non-functional requirements of the GrowWise platform, including performance, reliability, usability, reusability, and extensibility.

4.4.1 Performance

The system should help users learn efficiently and minimize the overall learning time.

4.4.2 Reliability

The website should be available at all times, so users can access learning paths and the AI mentor without interruption.

4.4.3 Usability

The platform should be simple to use. Users should quickly understand how to navigate and follow their learning path.

4.4.4 Reusability

The code should be modular, so it can be reused for new features or updates.

4.4.5 Extensibility

New learning paths, AI guidance, or project types should be easy to add in the future.

4.5 Assumptions

The following assumptions have been made for the GrowWise platform:

- Users have a web browser on their computer or device that supports JavaScript.
- An internet connection is required to access the platform.
- Users have basic understanding of using a website, like clicking buttons and navigating pages.
- Users have some basic knowledge of programming or development to follow the learning paths.
- Admins know how to create learning paths and manage AI-guided resources.

4.6 Use Cases

This section lists the use cases used in our application. The tables below provide detailed use cases.

4.6.1 User Login

Table 4.1: Use Case: User Login

Name	User Login		
Actors	Programmer, Developer		
Summary	User enters email and password, and after verification, is redirected to the dashboard.		
Pre-Conditions	User must have an account and must not be already logged in.		
Post-Conditions	User session is established, and dashboard is accessible.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	User opens login page	2	System displays email and password fields
2	User enters valid credentials	4	System verifies, creates session, redirects to dashboard
Alternative Flow			
3	User enters invalid credentials	4-A	System displays “Incorrect email or password” message

4.6.2 Track Selection

Table 4.2: Use Case: Track Selection

Name	Track Selection		
Actors	Programmer, Developer		
Summary	User selects a learning track to start personalized learning.		
Pre-Conditions	User must be logged in.		
Post-Conditions	Selected track is saved for learning path generation.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	User opens track selection page	2	System displays available tracks
2	User selects a track	4	System saves selection and confirms
Alternative Flow			
3	User does not select a track	4-A	System prompts: “Please select a track to continue”

4.6.3 Skill Assessment

Table 4.3: Use Case: Skill Assessment

Name	Skill Assessment		
Actors	Programmer, Developer		
Summary	User takes a test to check their current skill level.		
Pre-Conditions	User must select a track.		
Post-Conditions	Skill level is saved in the system.		
Special Reqs	Assessment covers technical and approach-based skills.		
Basic Flow			
Actor Action		System Response	
1	User starts assessment	2	System displays questions
2	User answers questions	4	System evaluates answers and calculates score
Alternative Flow			
3	User leaves assessment incomplete	4-A	System saves progress for later completion

4.6.4 AI Path Generation

Table 4.4: Use Case: AI Path Generation

Name	AI Path Generation		
Actors	Programmer, Developer		
Summary	System generates a personalized learning path based on assessment results.		
Pre-Conditions	Skill assessment must be completed.		
Post-Conditions	Personalized path is available for user.		
Special Reqs	Path should focus on approach-oriented learning.		
Basic Flow			
Actor Action		System Response	
1	System analyzes user score	2	Generates adaptive learning path
2	System displays path	4	User can start modules
Alternative Flow			
3	System cannot generate path	4-A	System shows message: “Try assessment again or contact support”

4.6.5 Agentic RAG Assistance

Table 4.5: Use Case: RAG AI Assistance

Name	RAG AI Assistance		
Actors	Programmer, Developer		
Summary	Users ask questions to AI assistant at any step for help.		
Pre-Conditions	User must be following a module.		
Post-Conditions	User receives guidance or answers from AI.		
Special Reqs	AI must provide context-aware answers.		
Basic Flow			
Actor Action		System Response	
1	User asks a question	2	System retrieves answer from knowledge base
2	User reads answer	4	System logs interaction for feedback
Alternative Flow			
3	AI cannot answer	4-A	System suggests reference material or next steps

4.6.6 Project-Based Assignment

Table 4.6: Use Case: Project Assignment

Name	Project Assignment		
Actors	Programmer, Developer		
Summary	AI assigns a project after completing modules, simulating a senior engineer.		
Pre-Conditions	Required modules must be completed.		
Post-Conditions	User submits project, system evaluates approach and understanding.		
Special Reqs	Evaluation focuses on problem-solving and approach.		
Basic Flow			
Actor Action		System Response	
1	System assigns project	2	User receives instructions
2	User submits project	4	System evaluates and provides feedback
Alternative Flow			
3	User cannot complete project	4-A	System allows resubmission with guidance

4.6.7 Admin Login

Table 4.7: Use Case: Admin Login

Name	Admin Login		
Actors	Admin		
Summary	Admin logs in to manage learning paths and RAG knowledge base.		
Pre-Conditions	Admin account exists.		
Post-Conditions	Admin can access dashboard.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	Admin opens login page	2	System shows email and password fields
2	Admin enters valid credentials	4	System grants dashboard access
Alternative Flow			
3	Admin enters invalid credentials	4-A	System shows error message

4.6.8 Manage Learning Paths

Table 4.8: Use Case: Learning Path Management

Name	Learning Path Management		
Actors	Admin		
Summary	Admin creates or updates learning paths for all tracks.		
Pre-Conditions	Admin must be logged in.		
Post-Conditions	Updated paths are available for users.		
Special Reqs	Paths must follow approach-oriented methodology.		
Basic Flow			
Actor Action		System Response	
1	Admin opens path management	2	System displays existing paths
2	Admin adds/edits path	4	System saves updates
Alternative Flow			
3	Admin enters invalid info	4-A	System prompts for correction

4.6.9 Manage RAG Knowledge Base

Table 4.9: Use Case: RAG Knowledge Management

Name	RAG Knowledge Management		
Actors	Admin		
Summary	Admin adds or updates resources for AI assistant.		
Pre-Conditions	Admin must be logged in.		
Post-Conditions	AI knowledge base is updated and ready for queries.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	Admin opens RAG management	2	System displays current resources
2	Admin adds/updates content	4	System updates knowledge base
Alternative Flow			
3	Content fails to save	4-A	System shows error message

4.6.10 Monitor User Progress

Table 4.10: Use Case: User Progress Monitoring

Name	User Progress Monitoring		
Actors	Admin		
Summary	Admin views user progress and completion of modules and projects.		
Pre-Conditions	Admin must be logged in.		
Post-Conditions	Admin sees real-time analytics.		
Special Reqs	Data must be accurate and updated.		
Basic Flow			
Actor Action		System Response	
1	Admin opens analytics page	2	System displays progress charts
2	Admin filters by track/user	4	System updates view
Alternative Flow			
3	Data fails to load	4-A	System shows message: “Try again later”

4.6.11 Learning Path Module Access

Table 4.11: Use Case: Module Access

Name	Module Access		
Actors	Programmer, Developer		
Summary	User follows the AI-generated path, accessing modules step by step.		
Pre-Conditions	User must have a generated path.		
Post-Conditions	Progress is tracked, and modules are completed.		
Special Reqs	Modules should be interactive and practical.		
Basic Flow			
Actor Action		System Response	
1	User opens module	2	System displays content
2	User completes module	4	System marks completion and unlocks next module
Alternative Flow			
3	User skips module	4-A	System allows revisiting later and reminds user

4.7 Hardware and Software Requirements

List the hardware and software requirements that will be required to develop and deploy the project.

4.7.1 Hardware Requirements

The system requires the following hardware components to ensure smooth operation and accessibility:

- Computer: A standard desktop or laptop computer capable of running a modern web browser.
- Internet Connection: A stable internet connection is required to access the platform and interact with AI-based features.
- AWS Virtual Private Server (VPS): The backend of the application will be hosted on an AWS Virtual Private Server to ensure scalability, performance, and secure data processing.

4.7.2 Software Requirements

The system requires the following software components:

- FastAPI
- Next.js
- OpenAI or Gemini API
- Supabase (PostgreSQL)
- AWS (Virtual Private Server)
- Docker
- Git and GitHub/GitLab
- Python 3.10+
- Node.js 18+
- VS Code
- Postman

4.8 Graphical User Interface

This section presents the graphical interface of the GrowWise platform. Each figure shows how the user interacts with different parts of the system from logging in, choosing a learning path, and taking assessments to viewing AI-generated evaluations and insights.

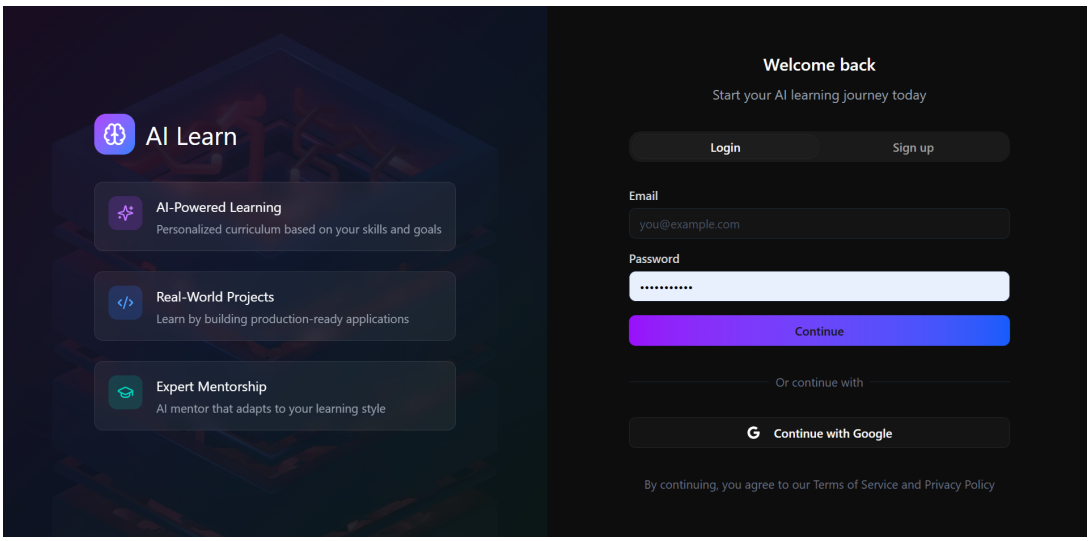

Figure 4.1: Login screen for users to enter credentials and access the system.

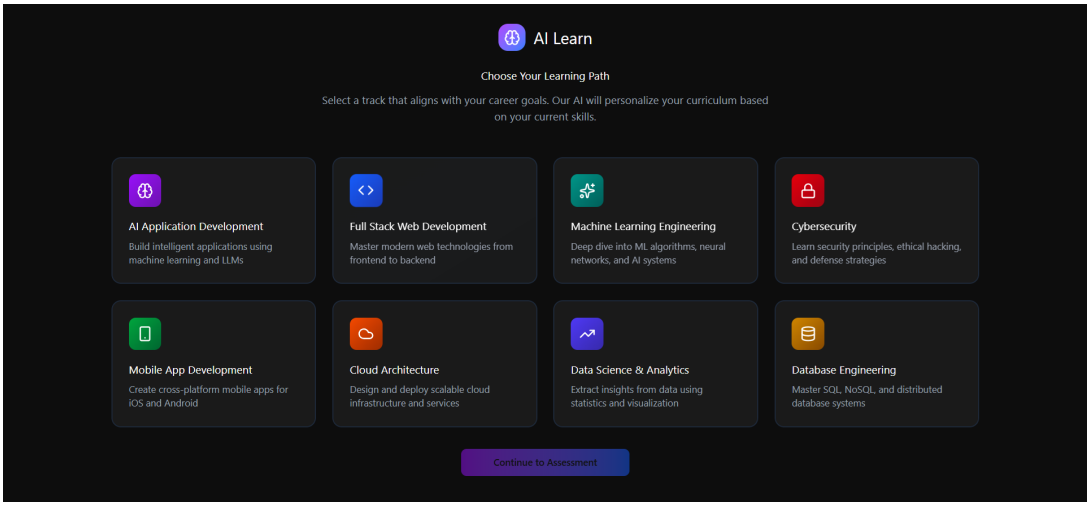

Figure 4.2: Track selection screen where users choose their learning path.

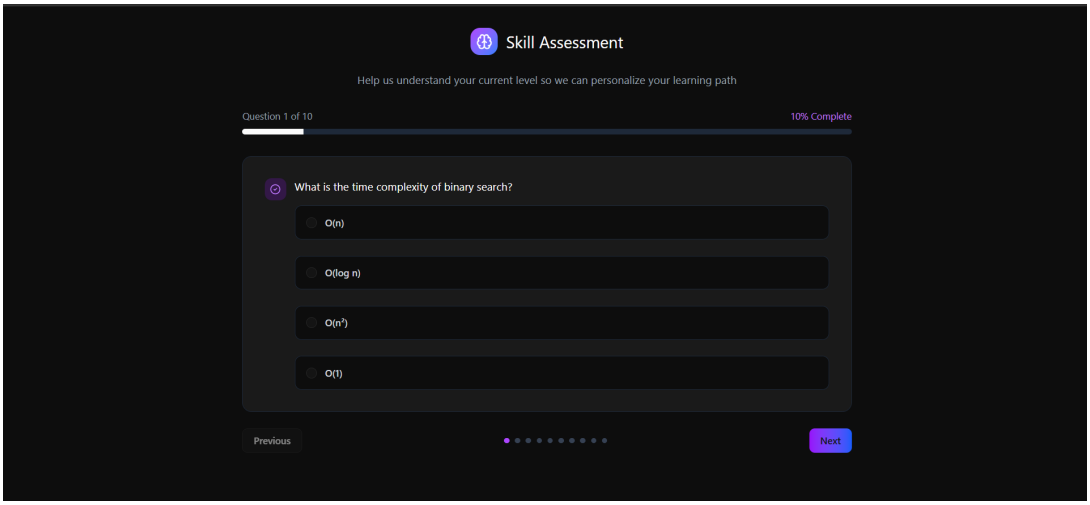

Figure 4.3: Skill assessment page for evaluating users' technical and problem-solving abilities.

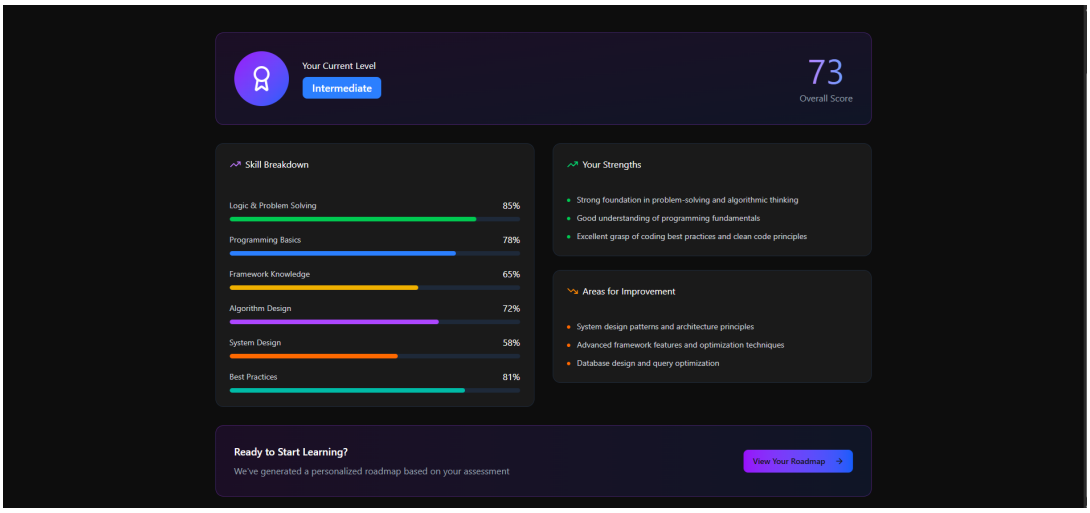

Figure 4.4: Assessment report showing the user's skill level and performance summary.

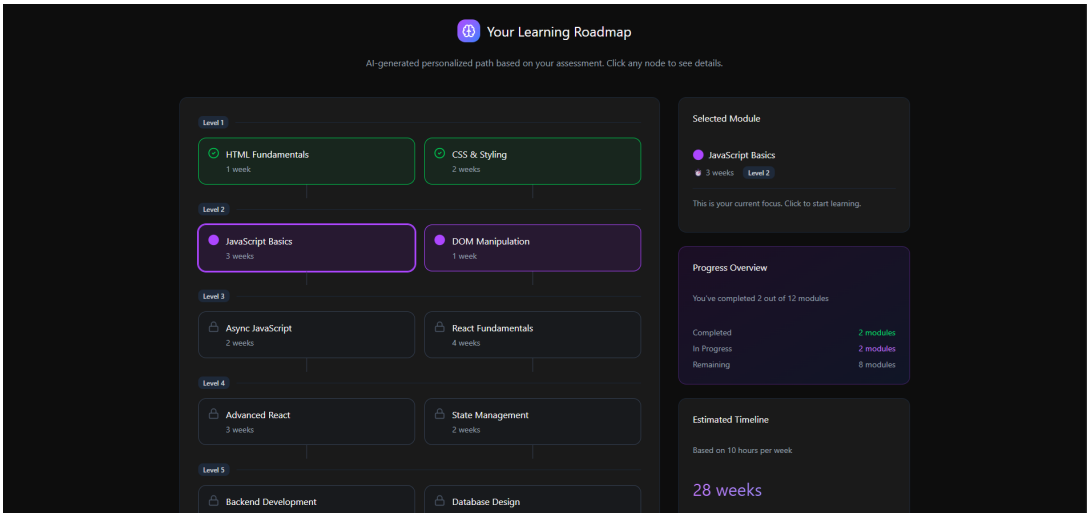

Figure 4.5: Personalized roadmap showing the AI-generated learning journey and module sequence.

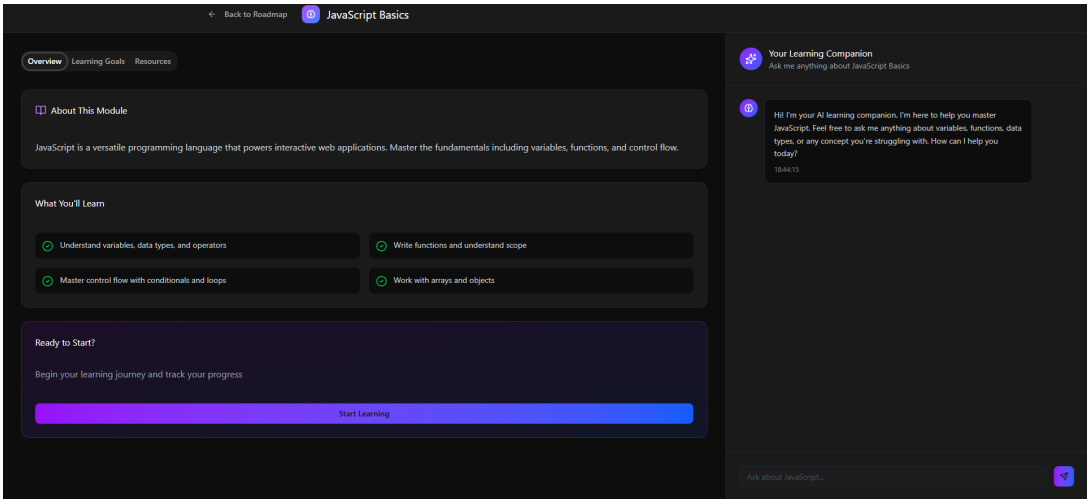

Figure 4.6: Learning module interface with study content and an integrated AI chatbot.

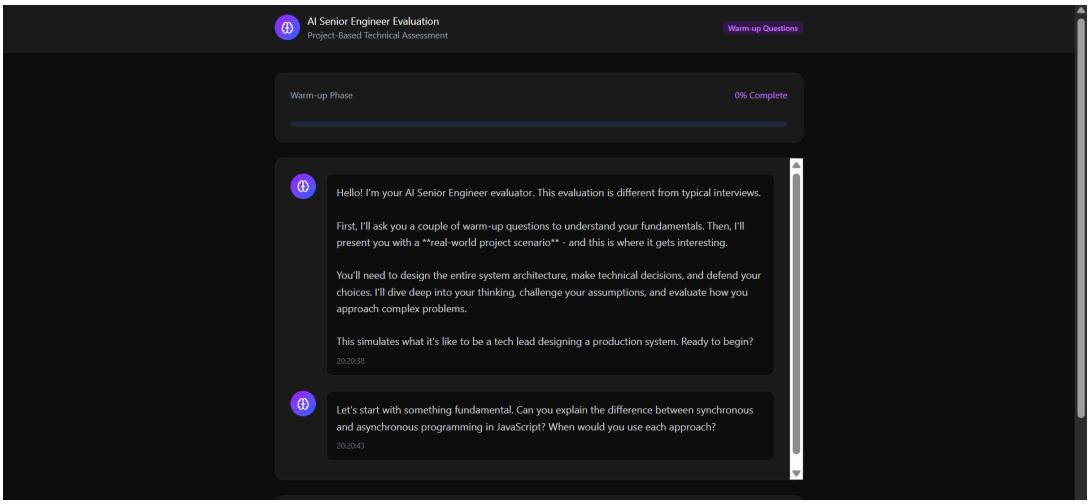

Figure 4.7: AI software engineer screen that assigns projects and evaluates user responses.

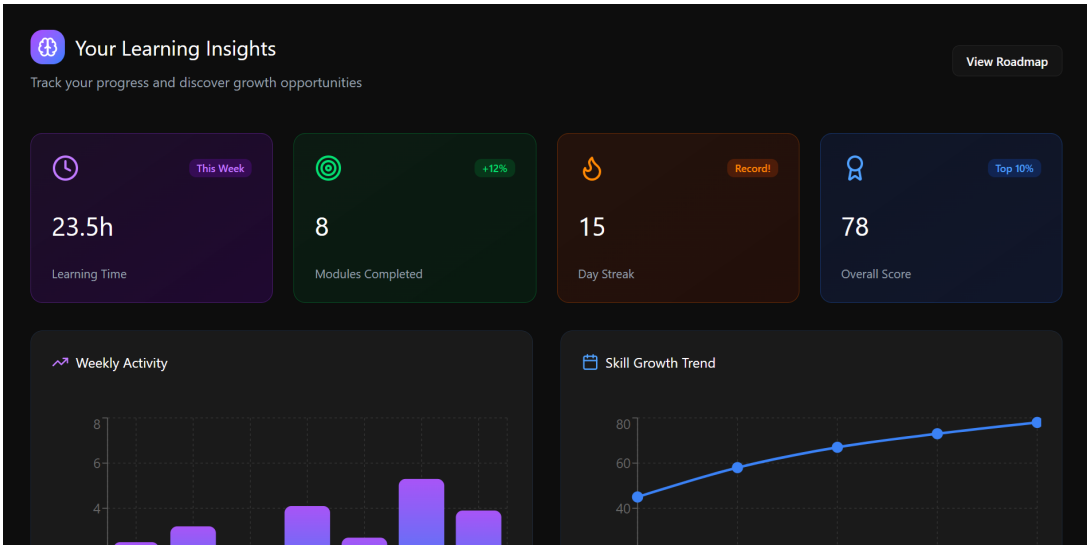

Figure 4.8: Insights dashboard with user progress analytics.

Figure 4.9: Advanced insights with AI evaluation and performance graphs.

4.9 Database Design (if required; this means if you are using noSQL, you will not provide ER but the other design element and data dictonary should still be there. Explain and elaborate your DB design)

4.9.1 ER Diagram

The GrowWise platform utilizes PostgreSQL (Supabase) as its primary relational database management system, enhanced with vector support (pgvector extension) to enable efficient storage and retrieval of embeddings for the RAG-based AI mentor functionality. The database design follows a normalized

relational model organized into four main functional modules: User Management & Authentication, Learning Track System, Progress & Assessment with AI Services, and Project Workflow & Evaluation.

The Entity-Relationship (ER) diagram presented below illustrates the complete database schema, showing all entities, their attributes, primary and foreign key relationships, and cardinality constraints. The design ensures data integrity through proper normalization, supports scalability for multiple concurrent users, and facilitates complex queries required for personalized learning path generation and AI-driven insights.

The database architecture has been designed to support three complementary PlantUML views that represent different aspects of the system: View 1 focuses on the user journey and progress tracking, View 2 emphasizes AI services and knowledge management, and View 3 details the project workflow and evaluation processes. These views correspond to the PlantUML diagrams (growwise_view1_user_journey.puml, growwise_view2_ai_services.puml, and growwise_view3_projects.puml) that provide visual representations of the database structure from different functional perspectives.

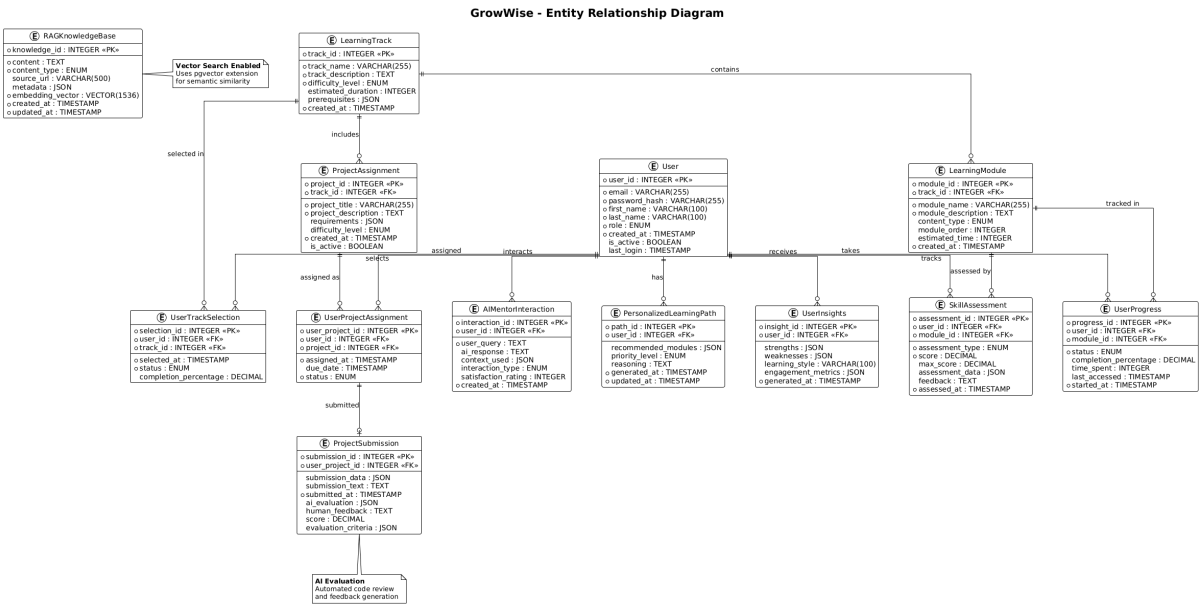

Figure 4.10: GrowWise Entity-Relationship Diagram with all entities, keys, and relationships.

Key Database Entities and Relationships:

- User Management: The *User* entity serves as the central anchor, maintaining one-to-many relationships with UserTrackSelection, UserProgress, SkillAssessment, PersonalizedLearningPath, UserInsights, AIMentorInteraction, and UserProjectAssignment entities.
- Learning Content Structure: The *LearningTrack* entity has a composition relationship with *LearningModule* (one-to-many), where each track contains multiple ordered modules. Tracks also relate

to UserTrackSelection, SkillAssessment, PersonalizedLearningPath, and ProjectAssignment entities.

- Progress Tracking: *UserProgress* links users to specific learning modules, tracking completion status and progress percentages. *SkillAssessment* connects users and tracks to evaluate proficiency levels, which then inform the generation of personalized learning paths.
- AI Services: *RAGKnowledgeBase* stores module-specific content with vector embeddings for semantic search, supporting the *AIMentorInteraction* entity that records all AI-powered Q&A sessions. *PersonalizedLearningPath* maintains AI-generated learning sequences based on assessment results.
- Project Management: *ProjectAssignment* defines reusable project templates linked to tracks. *UserProjectAssignment* creates individual instances for users, and *ProjectSubmission* stores user work along with AI and human evaluation data.

The complete ER diagram showing all entities, attributes, primary keys (PK), foreign keys (FK), and relationship cardinalities is visualized in the three PlantUML diagrams referenced above, which together provide a comprehensive view of the GrowWise database architecture.

4.9.2 Data Dictionary

The data dictionary provides a comprehensive specification of all database tables, their attributes, data types, constraints, and descriptions. This section documents the complete schema for the GrowWise platform, organized by functional modules to facilitate understanding and maintenance. Each table entry includes the attribute name, its data type, applicable constraints (such as primary keys, foreign keys, uniqueness, and null constraints), and a clear description of its purpose and usage within the system.

Table 4.12: Test case for User

Attribute Name	Data Type	Constraints	Description
user_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each user
email	VARCHAR(255)	UNIQUE, NOT NULL	User's email address for login and communication
password_hash	VARCHAR(255)	NOT NULL	Hashed password for secure authentication
first_name	VARCHAR(100)	NOT NULL	User's first name
last_name	VARCHAR(100)	NOT NULL	User's last name
user_type	ENUM('learner', 'admin')	NOT NULL, DEFAULT 'learner'	Role-based user classification
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account status flag

Table 4.13: Test case for LearningTrack

Attribute Name	Data Type	Constraints	Description
track_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each learning track
track_name	VARCHAR(200)	NOT NULL	Name of the learning track (e.g., "Web Development", "AI/ML")
description	TEXT	NULL	Detailed description of track goals and curriculum
difficulty_level	ENUM('beginner', 'intermediate', 'advanced')	NOT NULL	Overall difficulty rating of the track
estimated_duration_hours	INTEGER	NOT NULL	Approximate time required to complete the track

Table 4.14: Test case for LearningModule

Attribute Name	Data Type	Constraints	Description
module_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each module
track_id	INTEGER	FK(LearningTrack), NOT NULL	Reference to parent learning track
module_name	VARCHAR(200)	NOT NULL	Name of the learning module
module_order	INTEGER	NOT NULL	Sequence position within the track
content_type	ENUM('video', 'text', 'interactive', 'quiz')	NOT NULL	Type of learning content
estimated_duration_minutes	INTEGER	NOT NULL	Expected completion time for this module
learning_objectives	TEXT	NULL	Specific learning goals for this module

Table 4.15: Test case for UserTrackSelection

Attribute Name	Data Type	Constraints	Description
selection_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each selection
user_id	INTEGER	FK(User), NOT NULL	Reference to the user making the selection
track_id	INTEGER	FK(LearningTrack), NOT NULL	Reference to the selected track
selected_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	When the track was selected
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether this selection is currently active

Table 4.16: Test case for UserProgress

Attribute Name	Data Type	Constraints	Description
progress_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each progress record
user_id	INTEGER	FK(User), NOT NULL	Reference to the user
module_id	INTEGER	FK(LearningModule), NOT NULL	Reference to the learning module
status	ENUM('not_started', 'in_progress', 'completed')	NOT NULL, DEFAULT 'not_started'	Current completion status
started_at	TIMESTAMP	NULL	When the user began this module
completed_at	TIMESTAMP	NULL	When the user completed this module
progress_percentage	DECIMAL(5,2)	NOT NULL, DEFAULT 0.00	Percentage of module completed (0-100)

Table 4.17: Test case for SkillAssessment

Attribute Name	Data Type	Constraints	Description
assessment_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each assessment
user_id	INTEGER	FK(User), NOT NULL	Reference to the assessed user
track_id	INTEGER	FK(LearningTrack), NOT NULL	Track being assessed
total_questions	INTEGER	NOT NULL	Total number of assessment questions
score_percentage	DECIMAL(5,2)	NOT NULL	User's score (0-100)
assessment_data	JSON	NULL	Detailed question-by-question results
completed_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Assessment completion time

Table 4.18: Test case for PersonalizedLearningPath

Attribute Name	Data Type	Constraints	Description
path_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each learning path
user_id	INTEGER	FK(User), NOT NULL	Reference to the user
track_id	INTEGER	FK(LearningTrack), NOT NULL	Reference to the learning track
assessment_id	INTEGER	FK(SkillAssessment), NULL	Assessment that generated this path
path_data	JSON	NOT NULL	AI-generated sequence of modules and milestones
status	ENUM('active', 'completed', 'archived')	NOT NULL, DEFAULT 'active'	Current path status
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Path generation timestamp

Table 4.19: Test case for UserInsights

Attribute Name	Data Type	Constraints	Description
insight_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each insight
user_id	INTEGER	FK(User), NOT NULL	Reference to the user
insight_type	ENUM('strength', 'weakness', 'recommendation', 'milestone')	NOT NULL	Category of insight
insight_data	JSON	NOT NULL	Detailed insight content and metrics
priority	ENUM('low', 'medium', 'high')	NOT NULL, DEFAULT 'medium'	Importance level of the insight
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Insight generation timestamp

Table 4.20: Test case for RAGKnowledgeBase

Attribute Name	Data Type	Constraints	Description
knowledge_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each knowledge entry
module_id	INTEGER	FK(LearningModule), NULL	Associated learning module
content_type	ENUM('documentation', 'example', 'explanation', 'reference')	NOT NULL	Type of knowledge content
title	VARCHAR(300)	NOT NULL	Title or heading of the knowledge entry
content	TEXT	NOT NULL	Full text content for retrieval
metadata	JSON	NULL	Additional structured information
embedding_vector	VECTOR(1536)	NOT NULL	Vector embedding for semantic search (pgvector)
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether this entry is currently searchable
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Entry creation timestamp

Table 4.21: Test case for AIMentorInteraction

Attribute Name	Data Type	Constraints	Description
interaction_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each interaction
user_id	INTEGER	FK(User), NOT NULL	Reference to the user asking the question
module_id	INTEGER	FK(LearningModule), NULL	Context module if applicable
user_question	TEXT	NOT NULL	User's original question
ai_response	TEXT	NOT NULL	AI-generated response
context_data	JSON	NULL	Retrieved RAG context and sources
response_time_ms	INTEGER	NULL	Response generation time in milliseconds
feedback_rating	INTEGER	NULL, CHECK (feedback_rating BETWEEN 1 AND 5)	User satisfaction rating (1-5)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Interaction timestamp

Table 4.22: Test case for ProjectAssignment

Attribute Name	Data Type	Constraints	Description
project_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each project template
track_id	INTEGER	FK(LearningTrack), NOT NULL	Associated learning track
project_title	VARCHAR(300)	NOT NULL	Name of the project
project_description	TEXT	NOT NULL	Detailed project requirements and objectives
requirements	JSON	NOT NULL	Technical and functional requirements
difficulty_level	ENUM('beginner', 'intermediate', 'advanced')	NOT NULL	Project complexity level
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Project creation timestamp
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether project is currently assignable

Table 4.23: Test case for UserProjectAssignment

Attribute Name	Data Type	Constraints	Description
user_project_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each user assignment
user_id	INTEGER	FK(User), NOT NULL	Reference to the assigned user
project_id	INTEGER	FK(ProjectAssignment), NOT NULL	Reference to the project template
assigned_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Assignment timestamp
due_date	TIMESTAMP	NULL	Project deadline
status	ENUM('assigned', 'in_progress', 'submitted', 'evaluated')	NOT NULL, DEFAULT 'assigned'	Current project status

Table 4.24: Test case for ProjectSubmission

Attribute Name	Data Type	Constraints	Description
submission_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each submission
user_project_id	INTEGER	FK(UserProjectAssignment), NOT NULL	Reference to the user's project assignment
submission_data	JSON	NOT NULL	Project files, links, and deliverables
submission_text	TEXT	NULL	User's written explanation or documentation
submitted_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Submission timestamp
ai_evaluation	JSON	NULL	AI-generated evaluation results and feedback
human_feedback	TEXT	NULL	Optional instructor or admin feedback
score	DECIMAL(5,2)	NULL	Final project score (0-100)
evaluation_criteria	JSON	NULL	Rubric and evaluation breakdown

4.10 Risk Analysis

This part tell possible problem in project and how we fix them.

4.10.1 Technical Risks

System can got bug, server can crash or AI give wrong answer. Also OpenAI or Gemini API can change or stop work. To fix we will check everything and got backup plan ready.

4.10.2 Business Risks

Many other learning app already there so get user hard. API and server money can become too much. Our app different and good so people like it. We watch cost close.

Second risk is content quality.

AI can give wrong or old info. Sometime it say extra thing no need. To fix admin will check and change content many time.

4.10.3 User Risks

Some user find app hard or get bored. To fix we make design easy and nice. AI helper guide user step by step.

4.10.4 Security Risks

User login and share info. If no good security data can lost. We use safe server, strong password and protect all user thing.

Conclusion

In this chapter we clear all big info about GrowWise system. It tell main thing, how user and admin use system and how every part work. Functional and non-functional need write good so GrowWise fast, strong and easy use. Quality like speed, easy use and can grow big talk about for make user happy. Chapter also show assume, hardware software need and possible problem in make. Plus it got use case, screen design and database to show how module connect and work together. All this give full plan for build, test and make better the GrowWise learning app.

Chapter 5 High-Level and Low-Level Design

This chapter tell high level and low level design of our project.

5.1 System Overview

GrowWise is smart AI learning app that make study personal for developer. It check learner skill now, make special path for them, and give smart help with Agentic RAG teacher inside. The system care about teach how to think and do real project to make good problem solving skill.

5.1.1 Learning Management and Modules

GrowWise put content in small interactive lesson call learning node. Got clear goal, example and real task to do. User go step by step on AI make roadmap. Got check if you ready before next and auto track how you do.

5.1.2 Personalized Learning Paths

AI planner look your first test and what track you pick. Then make special roadmap only for you. When you study it change path auto from how fast you learn, what you good at and what need more time. So you not study same thing again or jump to too hard too quick.

5.1.3 Agentic RAG Mentor (Context-Aware Assistance)

You got smart AI teacher inside app. It find best resource for your lesson now and give easy explain right when you need. You ask anything it answer question, give clear tip and show helpful thing that match what you do now and what you did before.

5.1.4 Assessment and AI Software Engineer Evaluation

You do quiz, small task and real project challenge. One fake AI engineer check your work same like senior boss. It look your choice, good bad thing and code quality. Then give clear feedback with idea how to better.

5.1.5 Progress Insights and Feedback

Your own dashboard show how you do: how fast you go, what you good at and what still bad. This help you work on weak part, tell next step and make your plan fit you perfect.

5.1.6 High-Level Technology View

App build with new clean tool: Next.js for nice front look, FastAPI for quick back end, special AI part with LLM and RAG, and PostgreSQL from Supabase for save all thing. All service in Docker box and run on AWS server. Got good login and power control for safe.

5.2 Design Considerations

This part talk about main assume, limit and rule that make GrowWise design choice.

5.2.1 Assumptions and Dependencies

Related software and hardware

- AI service: We depend on good LLM API like OpenAI or Gemini and embedding for RAG. Got normal speed and use limit.
- Data layer: PostgreSQL from Supabase with vector add for search. Plus place for file and picture.
- Hosting: AWS VPS Linux x86-64 with Docker box. No GPU now but maybe later local think.
- Client: New browser like Chrome Firefox Safari with JavaScript on.

Operating systems

- Server: Linux for real use. CI/CD run in Linux box.
- Developer computer: Windows 10 or 11, macOS or Linux. All got Docker Desktop.
- Runtime base: Python 3.10 or more, Node.js 18 or more, PostgreSQL 14 or more.

End-user characteristics

- Main people: Junior developer who know little code and read English easy.
- Many user got slow or bad internet. App work no internet when can and start again no problem.
- Easy use for all: Keyboard move, good color, screen reader ok.

Possible and probable changes in functionality

- Maybe change AI company or model. So system got plug adapter and smart try again.
- New lesson and stuff add later. So database and tool made easy to grow.
- Test rule and feedback change time time. So check pipe got version.

5.2.2 General Constraints

- Hardware or Software Environment: App run on small cloud server like AWS or Render. Got only little CPU and RAM so we make everything light and quick. Database and vector place can handle normal amount content and embedding. No GPU in our place. All AI call go to outside like OpenAI or Gemini.
- End-User Environment: Work in any new browser on computer laptop or phone. Still ok on slow internet or little stop. Progress save by itself. Design change for every screen size. Look nice feel good everywhere.
- Standards Compliance: We do good privacy. Only keep what need and make it safe. Make for everyone can use: good color, easy read font, keyboard work all. Security follow OWASP rule for login and API key.
- Interface / Protocol Requirements: Everything talk with HTTPS and clean JSON REST API. Session use safe JWT token. Call to outside API got time limit and try again if fail.
- Security Requirements: All data encrypt from start to end with HTTPS. Password and token hash good and protect. Admin and student got different power with role control.
- Performance Requirements: Database search fast for many user same time. AI thing like make plan or check scale good when more student come.

5.2.3 Goals and Guidelines

- KISS (Keep It Simple): Make everything easy and not complicated. Less hard thing mean we build fast and got less bug.
- Easy-to-Use, Friendly Interface: App must feel simple and nice. Learner can use it without much help. Make them happy to keep going.

5.2.4 Development Methods

We build GrowWise with small part and agile way. This make system can change easy, grow big and easy fix.

- Modular Architecture: We cut everything to small alone piece. Like user login, make learning plan, AI check grade, and RAG helper. Each one we build test and fix alone.
- Agile Methodology (Scrum-inspired): We do short work time, show demo often, get feedback quick and always make small better.

- Backend Infrastructure: FastAPI make clean big REST back end. It talk easy with OpenAI, Gemini and other service.
- Frontend Implementation: Next.js + React make fast nice interface. Work good on phone computer everything.
- Data Persistence: PostgreSQL for normal data. Plus vector thing for smart search and RAG help.
- Version Management: GitHub (or GitLab) for work together, make branch and auto CI/CD pipe.
- Methodological Rationale: All this give us can change easy, keep long time and grow when AI or learning need change.

5.3 System Architecture

This section describes both the internal architecture of the modules and the external architecture of the system with other systems. A diagrammatic architecture is provided to illustrate the system's structure and interactions.

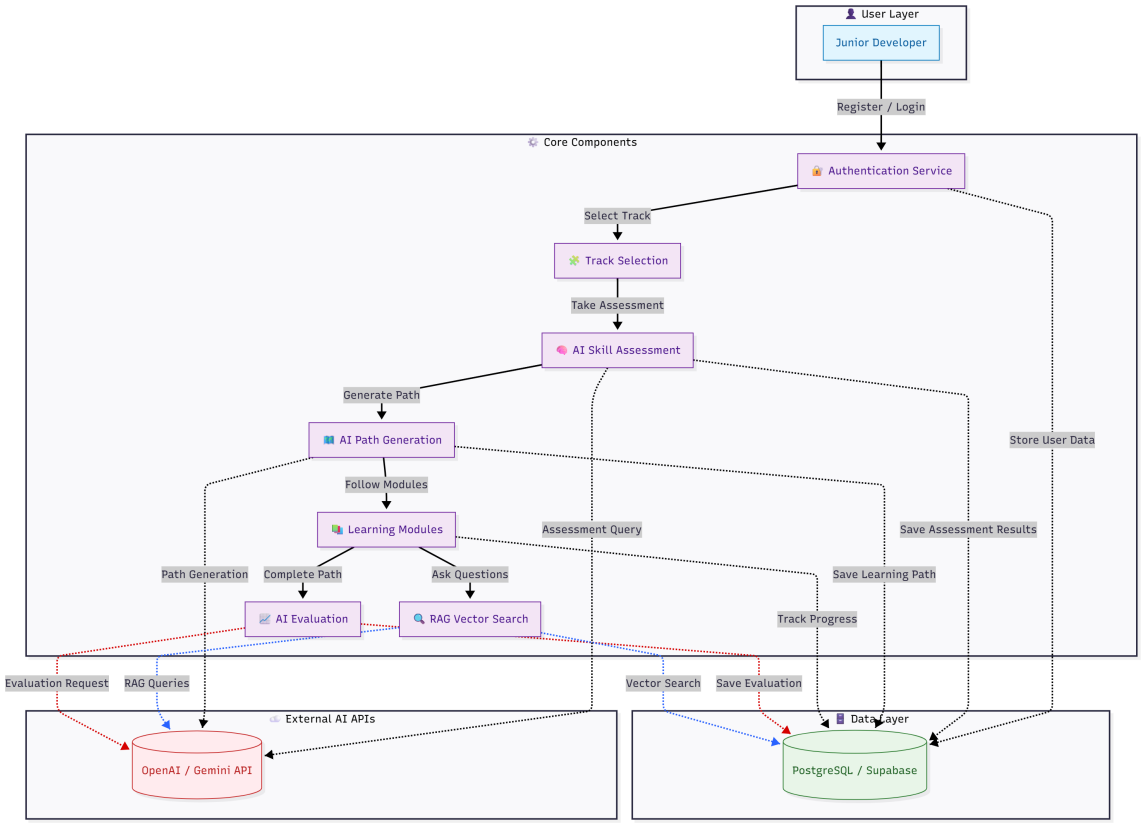

Figure 5.1: GrowWise architecture overview—showing interaction between frontend, backend, AI services, and database.

5.3.1 High-Level System Decomposition

The GrowWise platform breaks down into four main parts, each handling its own tasks:

- User Interface Layer: Takes care of all interactions with users and offers an easy-to-use web interface
- Application Services Layer: Holds the main business rules and guides user activities
- AI Services Layer: Deals with all AI features like testing skills, creating learning plans, and checking progress
- Data Persistence Layer: Manages storing, getting, and organizing data

5.3.2 Major System Responsibilities

The software system needs to handle these main jobs:

1. User Management: Safe login, profile handling, and keeping track of sessions
2. Adaptive Learning: Checking user skills on the fly and making custom learning plans
3. Content Delivery: Showing interactive lessons with built-in AI help
4. Intelligent Evaluation: Using AI to review user progress and project work
5. Progress Tracking: Keeping a full record and reports on how users are doing
6. Administrative Control: Managing lesson content, paths, and system settings

5.3.3 Component Roles and Responsibilities

5.3.3.1 User Interface Layer

- Authentication Module: Handles logging in, signing up, and managing sessions
- Track Selection Interface: Lets users pick their area of focus for learning
- Assessment Interface: Shows tests for skill checks and gathers answers
- Learning Dashboard: Displays custom learning plans and tracks progress
- Module Interface: Offers an interactive space for learning with an AI chat built in
- Evaluation Interface: Allows submitting projects and showing feedback

5.3.3.2 Application Services Layer

- User Service: Manages user accounts and login rules

- Assessment Service: Handles skill tests and scoring
- Path Generation Service: Builds custom learning maps
- Module Service: Takes care of delivering lessons and tracking progress
- Evaluation Service: Organizes AI reviews for projects
- Admin Service: Gives tools for admins to manage content

5.3.3.3 AI Services Layer

- LLM Integration Service: Connects to outside AI services like OpenAI or Gemini
- RAG System: Uses retrieval to add context for better AI help
- Vector Database: Keeps and searches embeddings of learning materials
- Assessment AI: Reviews answers and gives skill scores
- Path Planning AI: Makes flexible learning plans based on user info
- Evaluation AI: Gives feedback like a senior engineer on projects

5.3.3.4 Data Persistence Layer

- User Database: Stores user info, login details, and preferences
- Content Database: Handles lessons, paths, and learning resources
- Progress Database: Tracks how users are advancing, what's done, and performance
- Vector Store: Holds embeddings for quick content searches
- Session Storage: Manages short-term data and user sessions

5.3.4 Component Collaboration and Data Flow

The system components collaborate through well-defined interfaces and data flows:

1. User Authentication Flow: Interface Layer → User Service → Database
2. Assessment Process: Interface Layer → Assessment Service → AI Services → Database
3. Path Generation: Assessment Service → Path Planning AI → Module Service
4. Learning Process: Module Service → RAG System → Vector Database → AI Services
5. Evaluation Process: Evaluation Service → Assessment AI → Database

5.3.5 Architectural Rationale

This decomposition was chosen over alternative approaches for the following reasons:

- Layered Architecture: Provides clear separation of concerns and maintainability
- Microservices Pattern: Enables independent scaling and deployment of AI services
- API-First Design: Facilitates integration with external AI providers and future extensions
- Stateless Services: Ensures scalability and fault tolerance
- Modular AI Integration: Allows easy switching between AI providers and models

Alternative monolithic approaches were rejected due to scalability limitations and tight coupling between AI and business logic components.

5.3.6 External System Integration

The system integrates with the following external services:

- OpenAI/Gemini APIs: For LLM-based AI functionality
- AWS VPS: For hosting and infrastructure services
- Supabase: For database and authentication services
- Vector Database Services: For embedding storage and retrieval

5.3.7 Subsystem Architecture

The AI Services Layer is main part of system. It handle all smart AI thing by connect to outside API. It got many small part that work together make good learning.

5.3.7.1 AI Assessment Module

This module check user skill use outside AI API. It make question from path user choose and send answer to external AI check. It get back score and weak spot then save in database for make plan. All depend on external AI for make question and grade.

5.3.7.2 AI Path Generation Module

This module make special plan for every user with help from outside AI. It take test result and send to external API with user path. AI think and send back lesson order with step and goal. Then save plan in database so system can use.

5.3.7.3 Learning Module with RAG

This module give real learning with AI help from outside API. When user ask question or need help it send to external AI with lesson info. AI answer with good explain and tip. It also watch user progress and what they finish.

5.3.7.4 AI Evaluation Module

This module check user project use outside AI API. It send work and answer to external AI for look. AI give deep feedback, score and how to better. Like senior engineer talk, all from external AI.

5.3.7.5 External API Integration

All AI in system come from outside like OpenAI and Gemini. System call these API for test, plan, RAG help and project check. Every part handle own API talk, make request right, get answer and fix error. These API make system smart and fun.

5.3.7.6 Data Flow

User info go through AI layer, outside API do every step. Test answer go to API for score, plan request go to AI, help question in learning go to RAG external, project check by outside AI. All result save in local database for later and make system good.

5.4 Architectural Strategies

This part tell key choice we make for GrowWise design.

5.4.1 Using External AI APIs

We use outside AI like OpenAI and Gemini not make own AI. This way we get new AI easy no train hard. No need fix model or update ourself. Pay only when use so money save. Make own model too expensive and hard. Use external let us focus on learning app not AI.

5.4.2 Using RAG for Smart Help

We use RAG mean Retrieval-Augmented Generation for give learner good help. It take info from database and mix with AI for answer. This keep help new and right, no need update hand. Old fix page too boring and hard maintain. RAG give what need right time.

5.4.3 Choosing Modern Technology

We pick new tool like Next.js for front, FastAPI for back, PostgreSQL for data. These good with AI and easy use. Make platform fast and strong. Old tech slow and hard connect AI. New choice make build quick.

5.4.4 Using Microservices

We make system small separate service that talk each other. This let us change one without break all. If one problem other still work. Make system strong and easy handle.

5.4.5 API-First Design

We build everything with API first. This easy connect outside AI and add new thing later. Can make mobile app or connect other system. This way platform can change and ready for future.

5.5 Domain Model/Class Diagram

The domain model and class diagram show full how GrowWise build. It show main part, what they have, what they do and how connect. Setup split three view that help each other, each look different side how system work. View 1: User Journey & Progress talk how learner go through system. From sign up pick path, check skill, make plan, track progress and get report. This show user flow and how system change from result. View 2: AI Services & Knowledge Management show smart part for personal learning. Got RAG knowledge with vector, AI mentor for talk, path maker for plan change, RAG helper for lesson help. View 3: Project Workflow & Evaluation tell process for task and check. Show how project give to user, how submit and check by AI like engineer, feedback help better. This stress learn by real way and solve true problem. All view together show full GrowWise, how data move, user do, and AI fit good for personal learning.

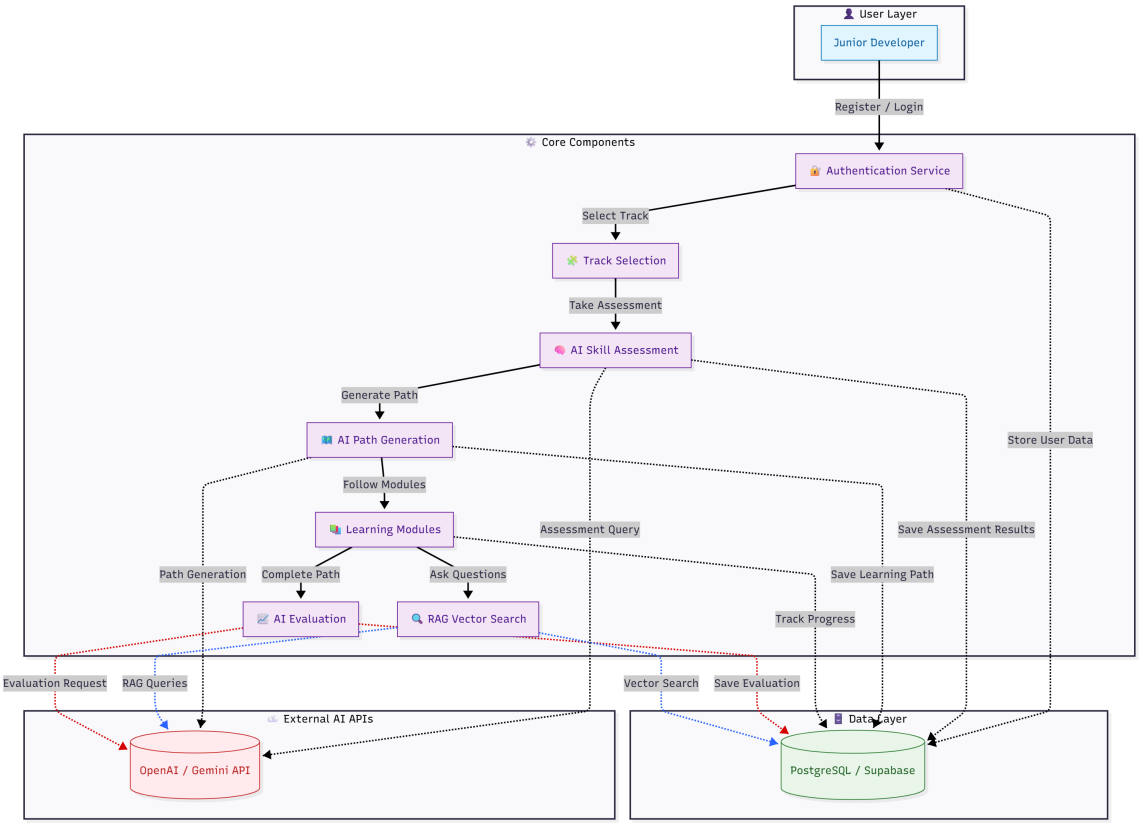

Figure 5.2: GrowWise system architecture: frontend, backend, AI, and database.

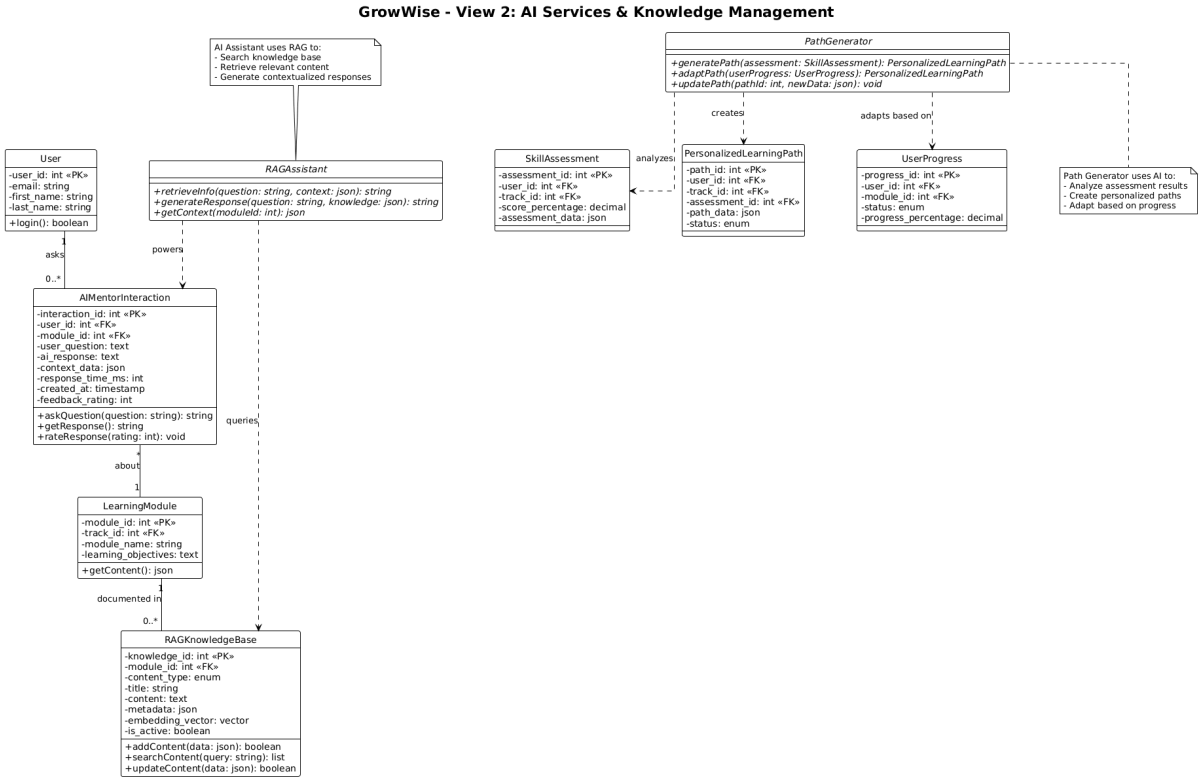

Figure 5.3: GrowWise – View 2: AI Services & Knowledge Management, highlighting components like the RAG knowledge base, contextual assistant, path generator, and AI mentor for adaptive, context-aware learning.

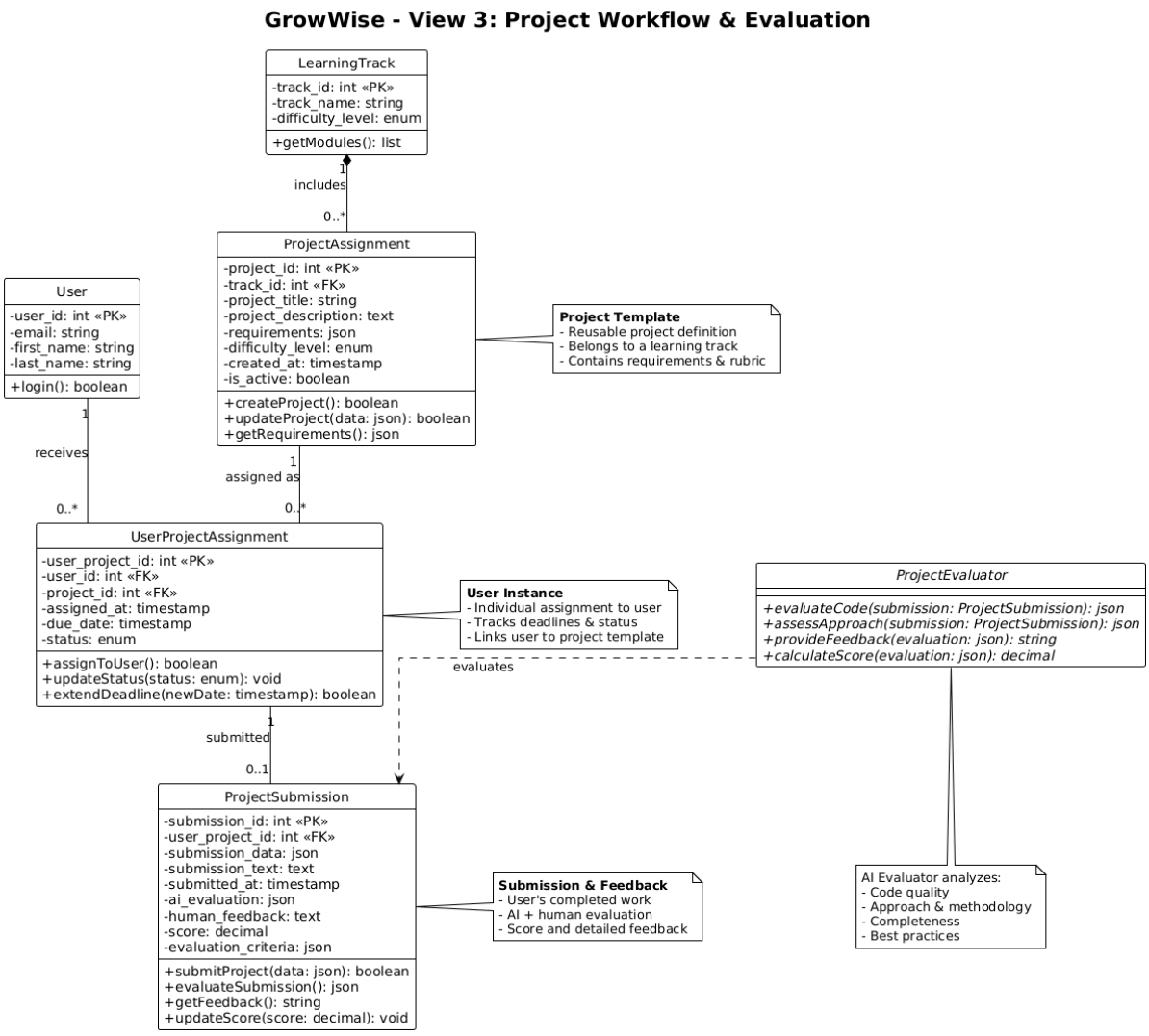

Figure 5.4: GrowWise – View 3: Project Workflow & Evaluation, showing how projects are assigned, submitted, and AI-evaluated with scoring and feedback.

5.6 Policies and Tactics

This section explains the exact design decisions and the implementation aspects that influence the work of the system.

5.6.1 Codebase Organization

Our code is well-organised in a simple folder structure. The primary project is divided into three major folders: frontend that contains the user-facing aspects, backend which contains server-related aspects, and shared which contains code which is shared between the two ends. We put similar files together just in the same folder. Similar, all of the code relating to logins in a single location, all lesson module code in a different location. In this manner, it is easy to locate things and update them. We use a regular name

of folders and files. Folders are small case with the use of underscores and files are similar to that. It assists in all getting the layout quickly.

5.6.2 Development Tools

We select specific instruments in constructing the application. VS Code is the primary choice of everyone because it is compatible with our languages. The version tracking is done by Git and the code is stored in GitHub. Docker deploys the app on local computers and in live environments to ensure that environments remain similar. The APIs are tested by Postman and the front end is debugged using Chrome DevTools. We also record everything so that new members can be on their feet fast. And automatic install and set up scripts.

5.6.3 Coding Guidelines

We possess simple coding regulations. Code must be well maintained and understandable. Instruments auto-make it appear homogenous. Classes and functions are assigned meaningful names. We add notes for tricky parts. In Python, type hints are used to explain things. Code before merge is auto checked to be compliant to rules.

5.6.4 Testing Strategies

There are various ways of testing code. Unit tests are analyzed of individual functions, the integration tests of how components are linked together and end-to-end tests of the complete user paths. Tests are made of new features and fixes. They have to die before they can unite. We monitor the extent of code tests and are ambitious. Automated applications will perform tests on all the changes and this will ensure that things are always working and that any problems are identified early.

5.6.5 Documentation Practices

We document everything well. The APIs receive clear definitions on the uses and how they work. Complex bits have code notes. We maintain platform user guides and dev guides of the code. In addition, deployment instructions and troubleshooting manuals. We update documentations with the changes and go through them regularly to ensure that they are accurate and useful.

5.7 Conclusion

This chapter provided a complete design specification of the GrowWise AI-based adaptive learning platform, including overall decisions and construction plans. It began with an in-depth analysis of what

the system requires and constrains which resulted in a scalable, expandable structure that addresses the difficulties in teaching developers. The proposed architecture consists of the latest technologies, such as Next.js as the front, FastAPI as the back-end services, PostgreSQL as the database, and AI APIs, as the intelligent path generator, and verification. It is based on learning approaches, where path making on the fly, AI guidance in real-time (through RAG) knowledge pulls, and complete checks analysing the skills as well as problem-solving methods are considered. The decisions are made according to growth, ease of maintenance and user feel and address security, speed and access needs. The modular design enables the part of the building and testing to be made in isolation, which can be improved and added easily. The comprehensive documentation, including class schema, system connections, and construction architecture provide a good groundwork to develop on. The design addresses the primary objectives of a bespoke, flexible platform that transcends predetermined courses in order to provide dynamic systems that are conscious and tailored to the requirements and learning styles of every developer.

Chapter 6 Implementation and Testing

The chapter addresses the process of creating and testing the GrowWise prototype. It contains the technology selections, determining the particulars of the keybuilds of the parts, the connections with AI, data archives, test arrangement, and test results, launch, and present constraints.

6.1 Implementation

The following are the construction specifications of the components and procedures of the plan.

6.1.1 Learning Track and Module System

The Learning Track and Module System establishes the framework of the simple system of GrowWise, with lessons arranged into definite sequences. Big tech areas are referred to as tracks and modules are the units within the tracks. To begin with, we have created a flexible database structure consisting of multitrack and ordered modules. The back end is a FastAPI SQLAlchemy based implementation of LearningTrack and LearningModule items. Songs are listed on a table named, described, difficulty, and time estimate. Tracks are interconnected through a key to a module and order exists. The next.js and React front end display tracks in cards which contain information such as description, difficulty, and time. The selection of a track stores a UserTrackSelection link. The module view presents information sequentially with text, videos, coding assignments, and quizzes. It verifies requirements to maintain the appropriate sequence. Modules are loaded depending on the user spot in the path. UserProgress in progress tracks including status, time and percent. Admins have the ability to revise content without discontinuing progress, quality versions.

6.1.2 AI-Powered Skill Assessment Module

The AI-Powered Skill Assessment Module tests to determine user knowledge and problem-solving to identify the optimal start. It combines smart grading with AI generated questions. The track is made by using outside AI APIs such as OpenAI or Gemini to make questions. The back end receives track information, constructs a prompt that the AI will use to generate questions on knowledge and methods. It establishes types, problems and coverage. There are questions that have options of facts, reasoning questions, and practice questions in terms of their coding. Every save has its track, difficulty and criteria. We retain a bank to be reused, and make others to personal checks. In grading, questions and answers are submitted to the AI using back end with the prompt to verify the correctness, reasoning and understanding. It identifies areas of weaknesses, strengths, and provides feedback on techniques.

Scores, JSON, and time of results are stored in SkillAssessment. AI information informs the decisions of making a path, closing holes, and creating strengths.

6.1.3 AI Learning Path Generation Module

The AI Learning Path Generation Module creates personalized road maps based on the assessments and creates the order of the best modules. It encompasses information analysis and artificial intelligence design. It starts after assessment. Results such as scores, gaps, strengths and track are obtained through back end. This passes on to AI with module data, requirements, challenges, and objectives. AI is the planner based on immediate information on the user, modules, connections and regulations. It tracks a route using systematized modules, areas of focus, times and milestones. Path It is a JSON containing module IDs, positions, prerequisites, times and objectives. It consists of adaptive features such as reviews, additional practice and advanced options. It Stores the PersonalizedLearningPath as a flexible JSON file. UserProgress System watches move forward in the path, performance check, and makes the path refined with AI when necessary. It monitors progress, speed and problem areas. Massive alterations cause path updates to align with actual learning. Path is displayed as interactive map on front end where the nodes represent modules and display the status. It has used percent, time remaining and milestones. Position, links, and module details are displayed to the users.

6.1.4 RAG Implementation for Integrated Learning Sources Context

The Retrieval-Augmented Generation (RAG) system is developed on the lines of the smart system that retrieves learning materials to form a knowledge base rich in context. This allows the AI mentor to offer precise and useful guidance in terms of well-selected educational content. This was constructed in a stepwise fashion that manages materials, constructs semantic embeddings, and mining out pertinent content intelligently. It begins with content importation, during which we collect information such as documentation, tutorials, code snippets, explanations and documentation and prepare it. The back end contains uploading file, picking type and linking to modules administrative tool. It works with PDF, markdown, text and JSON. The processing is a breaking down of docs into meaningful pieces. We cut smartly, but we still keep structure structured such as paragraphs, headers, code, sentences, etc. as opposed to cutting by size. Every work is converted to embeddings with the models of OpenAI such as text-embedding-ada-002 to obtain vectors that encode the meaning. This facilitates the process of locating the related concepts, not only the identical words. To enable us to find the vectors quickly in the database, we use PostgreSQL and pgvector. The pieces are stored in the RAGKnowledgeBase with their vectors, source information, type, link to a module, and some time. Indexes also enable rapid searches with large quantities of data. Retrieval combines semantic search and filters. In the case of user query,

we embed the query, and find similar vectors by cosine. We are able to narrow down to the module or type. We prioritize the best pieces, take the most few such as 3-5 and submit them along with the question to the LLM. The prompt will direct the AI to apply that information to give precise and cordial responses such as a mentor. We made our adjustments towards enhanced speed and precision. Caching is the records of frequent questions and descriptions so as to reduce the number of calls and accelerate the process and time-outs are used to refresh it. We extend queries with module background and user history also. All the mentor chats are logged in AIMentorInteraction, including the question, chunks, response, time and feedback. This assists in identifying patterns, missing knowledge and enhancing retrieval. Each module of the front end possesses a smooth chat that displays history, indicates natural questions and clear responses. It proposes questions, displays content that is referred to, and allows users to rate responses.

6.2 Conclusion

This chapter covered build decisions and tested how it works on various levels. The results demonstrate that a flexible learning flow supported by AI is achievable and preconditions further construction and development.

Chapter 7 Conclusions and Future Work

7.1 Project Summary and Achievements

The final project is the construction of GrowWise, an AI-enhanced adaptive learning platform, which seals huge gaps in the way developers learn. It struck the overall target of creating a personalized system centered on approaches, and passing the outdated fixed courses to deliver vibrant, conscious education tailored to each individual. In examining fifteen other platforms we identified such weaknesses as lack of real-time checks on structure, minimal emphasis on problem solving, and complete AI mentoring. The fix consists of Next.js, FastAPI, PostgreSQL, and AI API as the tools of a scalable and simple to maintain platform that develops skills and thought processes.

7.2 Key Findings and Results

The review demonstrated that others excel in areas such as projects (freeCodeCamp, Udacity), adaptive tests (Pluralsight, DataCamp), and AI (CYPHER Learning), but none of them combine it in the way GrowWise does. The arrangement closes the gaps with on-the-fly routes, real-time assistance with AI through RAG pulls, and complete abilities and logic examination. The modular construction allows parting work and tests, and the additions and amendments are not difficult. The complete documentation containing diagrams, interactions and plans provide proper building and rollout.

7.3 Project Scope Coverage and Objectives Achievement

We went through it all and achieved all objectives. The design manages users, path making, AI tests and tracking where necessary. It accompanies tracks, provides personal experiences and has user and admin views. All the needs concerning functionalities, performance, security, scale, and convenience are addressed. It is a combination of a method-based approach to learning and AI assistance, forming a new platform between school and work. Our fix was to the extent of what is allowed to ensure that developer learning issues were full.

7.4 Challenges Faced and Limitations

We hit some hurdles. Combining AI services and maintaining speed required special consideration and adjustments. Responsive design and backups were required to make it work well on devices and networks. The dynamic nature of AI content required powerful error corrections. Constant adjustments were balancing between deep checks and speed. There are limitations such as the use of external AI,

which might compromise the uptime and speed. It has gone web based and thus not much on mobile enthusiasts. Besides, its effectiveness is determined by the quality of materials and knowledge base.

7.5 Future Work and Recommendations

7.5.1 Phase 2 Development Plan (FYP-2)

A phase 2 development plan (FYP-2) is provided below. To complete the final section, the following is a proposal to release main works and promote the platform:

1. Project-Based Assessment and AI Evaluation: Assemble the entire project system to real software problems. This includes requirements and specifications interface. The back end is concerned with the start, to check and assigning through progress and skills. Make submissions on AI checks of code, structure, methods, patterns, and choices and provide feedback on good parts, fixes, suggestions, and tips. It assists in connection of learning to actual work with pro feedback of skills.

2. Progress Tracking and Insights Module: Have a complete analytics configuration to journey views. Add dashboard, charts, percentages, time, skill graphs and compares. Back end monitors actions, speeds, spots and weak points or strengths. The insights that AI examines include mastered concepts, requires further discussion, speed, and feedback. It is in line with averages of similar folks. This provides information to make decisions, remain motivated, and spend time efficiently.

3. Content Development: Prepare complete content, assignments and templates of tracks such as web, data science, and mobile. Make learning methods interactive.

4. Testing and Validation: Conduct lots of tests such as unit, integration, user and performance to ensure reliability and ease of modules.

5. User Interface Enhancement: develop responsive, accessible interfaces with good design to be evaluated and provide insights, which are usable on all devices.

6. Performance Optimization: Add caching, database tuning and AI optimizations to have high, scalable performance, in particular, checks and insights.

7. Deployment and Monitoring: Make it live and add watching, logs and analytics to maintain and improve.

7.5.2 Long-term Recommendations

In more detail, the following ideas can be noted:

- Mobile Application Development: Add iOS and Android applications to access it and have fun even better.
- Advanced AI Capabilities: Have finer AI run code checks, structure reviews, and custom tips that are oriented at learning the language.
- Community Features: Have peer reviews, forums and projects that are undertaken in groups.
- Industry Integration: Collaborate with companies in actual projects and mentoring.
- Advanced Analytics: Develop insights using predictions, gap forecasts, and general measures to keep improving.
- Multi-language Support: Support additional languages and structures to access additional developers.
- Enterprise Features: Custom paths and make team tools and tracking for companies and schools.

7.6 Final Remarks

GrowWise will contribute significantly to the developer education, sealing discontinuities with novel AI and emphasis on methods. The concrete construction and build strategy prepares a platform-altering platform that accommodates dynamically evolving technology. It demonstrates the ability of AI personal learning technology to transform technological learning, providing a scalable solution to students, institutions, and enterprises. The FYP-2 plan continues the momentum on a complete platform that improves the developer skills in the modern world.